

*PROYECTO:*

# Canela's Desk



*Realizado por:*

*Animal Team.*

- *Sofía Arcones Blanco.*
- *Dianira Hualpa Vásquez.*
- *César Augusto Negrín Figueredo.*



**Desarrollo de Aplicaciones Web (DAW)**

**Valladolid, a 04 de febrero de 2026**

|                                              |    |
|----------------------------------------------|----|
| Introducción.....                            | 4  |
| 1.1 Descripción del Proyecto .....           | 4  |
| 1.2 Finalidad y Necesidad Abordada .....     | 4  |
| 2 Análisis.....                              | 4  |
| 2.1 <i>Dataset</i> utilizado .....           | 4  |
| 2.2 Requisitos funcionales .....             | 5  |
| 2.3 Diagramas de casos de uso .....          | 6  |
| 2.3.1. Identificación de Actores.....        | 7  |
| 2.3.2. Descripción de los Casos de Uso ..... | 7  |
| 3 Diseño.....                                | 9  |
| 3.1 Modelo de base de datos.....             | 9  |
| 3.1.1 Diagrama Entidad-Relación: .....       | 9  |
| 3.2 Diseño de la interfaz .....              | 11 |
| 3.2.1 Prototipo.....                         | 11 |
| 3.2.2 Guía de estilos .....                  | 13 |
| 4 Desarrollo .....                           | 14 |
| 4.1 Stack tecnológico .....                  | 14 |
| 4.2 Estructura de carpetas .....             | 15 |
| 4.2.1 Estructura de Directorios.....         | 15 |
| 4.2.2 Descripción de Directorios Clave ..... | 15 |
| 4.3 Arquitectura del sistema .....           | 16 |
| 4.3.1 Ventajas de este enfoque: .....        | 16 |
| 4.3.2 Flujo de Datos .....                   | 16 |
| 4.3.3 Capa de Datos (Models) .....           | 16 |
| 4.4 Descripción del funcionamiento .....     | 23 |
| 4.4.1 Manual de usuario.....                 | 23 |
| 5 Pruebas.....                               | 34 |
| 5.1 Prueba de Usabilidad: .....              | 34 |
| 5.2 Prueba de Accesibilidad con WAVE: .....  | 38 |
| 5.2.1 Análisis de Navegación y UI .....      | 38 |

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 5.2.2 | 2. Evaluación de Accesibilidad (Test WAVE) .....        | 39 |
| 5.2.3 | Conclusión de Usabilidad .....                          | 39 |
| 5.3   | Testing .....                                           | 40 |
| 5.3.1 | Resumen de ejecución de pruebas.....                    | 40 |
| 5.3.2 | Metodología y entorno.....                              | 41 |
| 5.3.3 | Desglose de Pruebas por Módulo .....                    | 41 |
| 5.3.4 | Depuración y Validación del Esquema de Datos .....      | 42 |
| 5.3.5 | Conclusión del Proceso.....                             | 42 |
| 6     | Despliegue.....                                         | 42 |
| 6.1   | Instrucciones de instalación en local: .....            | 42 |
| 6.1.1 | Requisitos previos.....                                 | 43 |
| 6.1.2 | Pasos de instalación.....                               | 43 |
| 6.1.3 | Verificación del despliegue .....                       | 44 |
| 6.1.4 | Descripción de los archivos de configuración.....       | 44 |
| 6.1.5 | Resultado final: .....                                  | 45 |
| 6.1.6 | URLs: .....                                             | 45 |
| 7     | Sostenibilidad y "Green Coding" .....                   | 45 |
| 7.1   | Aplicación Sin desplegar .....                          | 45 |
| 7.1.1 | Estrategia de caché .....                               | 45 |
| 8     | Conclusiones.....                                       | 47 |
| 8.1   | Autoevaluación .....                                    | 47 |
| 8.1.1 | Dificultades encontradas: .....                         | 47 |
| 8.1.2 | Valoración de la metodología. ....                      | 48 |
| 8.2   | Líneas futuras.....                                     | 48 |
| 8.2.1 | Gamificación y Economía Interna (Sistema de Bayas)..... | 48 |
| 8.2.2 | Optimización de la Experiencia de Usuario (UX/UI) ..... | 49 |
| 8.2.3 | Funciones Sociales y Comunidad.....                     | 49 |
| 8.2.4 | Hacia un producto comercializable .....                 | 49 |
| 8.2.5 | Reflexión Final.....                                    | 50 |
| 9     | Bibliografía.....                                       | 51 |

**índice de tablas**

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| Tabla 1. Caso de uso de Critterpedia. ....                                                      | 8  |
| Tabla 2. Caso de uso de Ver Catálogos. ....                                                     | 8  |
| Tabla 3. Resumen de ejecución de pruebas. ....                                                  | 41 |
| Tabla 4 Estrategia de persistencia y TTL por tipo de dato .....                                 | 46 |
| Tabla 5 Comparativa de rendimiento: Sistema de archivos local frente a Redis en Docker<br>..... | 46 |
| Tabla 6 Análisis comparativo de consumo energético y emisiones de CO <sub>2</sub> .....         | 47 |

# Introducción

## 1.1 Descripción del Proyecto

Este proyecto consiste en el desarrollo de una aplicación web interactiva denominada "Canela's Desk", centrada en el ecosistema del videojuego *Animal Crossing: New Horizons*. La plataforma funciona como un gestor de colecciones personal y un centro de información en tiempo real para los jugadores.

La temática gira en torno a la catalogación de elementos biológicos y culturales (peces, insectos, criaturas marinas, fósiles y obras de arte). Para ello, se ha utilizado el conjunto de datos abierto proporcionado por ACNH API, una interfaz técnica que centraliza toda la información de los ítems del juego, incluyendo precios, estacionalidad, horarios de aparición y modelos visuales.

## 1.2 Finalidad y Necesidad Abordada

La aplicación nace para resolver una limitación intrínseca del juego original: la consulta de disponibilidad y progreso fuera de la consola.

El problema principal que aborda es la falta de una herramienta externa ágil que permita a los ciudadanos de la "isla" realizar las siguientes tareas sin interrumpir su partida:

- **Seguimiento de Colecciones:** Muchos usuarios olvidan qué ejemplares han donado ya al museo de Sócrates. Esta app permite marcar y visualizar de forma rápida el progreso total.
- **Gestión de la Estacionalidad:** Dado que la fauna del juego cambia según el mes y la hora real, la aplicación actúa como un asistente inteligente que filtra qué criaturas están disponibles para capturar en el momento exacto de la consulta.
- **Optimización de la Economía del Juego:** Al mostrar los precios de venta de forma clara y organizada, ayuda al usuario a priorizar qué capturas conservar para maximizar sus beneficios (bayas) dentro del juego.

En definitiva, la plataforma transforma un conjunto complejo de datos estáticos en una experiencia de usuario fluida y visual, mejorando la planificación y el disfrute del tiempo de ocio del jugador.

# 2 Análisis

## 2.1 Dataset utilizado

- **Nombre:** ACNHAPI Dataset (Alexis Lours).
- **URL:** <https://github.com/alexislours/ACNHAPI>

**Nota técnica:** Debido a la inestabilidad de los servicios externos, los datos se han obtenido en formato ficheros JSON desde el repositorio original, que cuenta con una Licencia MIT. Estos datos han sido procesados e integrados en una base de datos propia gestionada por nuestra propia API en Laravel para garantizar la soberanía y disponibilidad del servicio.

- **Campos utilizados:**

- **Generales:** ID, file-name, name-EUes (nombre en español), name-USen (nombre en inglés), icon\_uri e image\_uri.
- **Específicos de fauna (Bichos, peces y criaturas marinas):** availability, price (precio de venta), catch-phrase y museum-phrase.
- **Específicos de arte:** hasFake (indicador de autenticidad), museum-desc (descripción de Sócrates), buy-price y sell-price.
- **Específicos de fósiles:** price, museum\_phrase y part-of
- **Vecinos:** personality, species y birthday, gender, subtype, hobby, catch-phrase, bubble-color, text-color, saying.

## 2.2 Requisitos funcionales

El sistema "Canela's Desk" implementa las siguientes funcionalidades adaptadas a las necesidades del usuario:

- **Gestión de Usuarios y Personalización:**
  - Registro e inicio de sesión seguro para el acceso a funciones personalizadas.
  - Perfil de usuario editable: posibilidad de personalizar la información y estética del perfil del jugador.
- **Critterpedia (Catálogo Universal de Coleccionables):**
  - Consulta de la base de datos completa de bichos, peces, criaturas marinas, fósiles y obras de arte.
  - Fichas detalladas con información sobre estacionalidad, hábitat y descripciones.
  - Acción de donación: botón integrado en cada ítem del catálogo para registrar su entrega al museo.
- **Catálogo de Vecinos (Aldeanos) y Favoritos:**
  - Visualización de todos los aldeanos disponibles en el juego con sus atributos (especie, personalidad, cumpleaños).
  - Sistema de favoritos: cada vecino incluye un icono de corazón que permite al usuario marcarlo para añadirlo a su lista personal.
- **Gestión de "Mi Museo":**
  - Sección exclusiva que filtra y muestra únicamente los ejemplares que el usuario ha marcado previamente como donados en la Critterpedia.

- Panel de residentes: visualización de los aldeanos favoritos seleccionados por el usuario.
- **Asistente y Eventos en Tiempo Real:**
  - Algoritmo de disponibilidad: Detección automática de criaturas activas según el mes y la hora actual del sistema.
  - Sistema de Efemérides (Caja de Regalo): Componente interactivo en el inicio que detecta si es el cumpleaños de algún aldeano. Al interactuar con la caja de regalo, esta se abre para mostrar una felicitación especial al vecino cumpleañosero.
- **Ambientación Musical Dinámica:**
  - Reproductor de audio integrado que sincroniza la música de fondo con la hora real, emulando la banda sonora cambiante del videojuego original.
- **Visualización de Progreso:**
  - Panel de estadísticas con gráficas interactivas que muestran el porcentaje de completitud de cada ala del museo (peces, bichos, fósiles, etc.).
  - Interfaz optimizada con paginación y navegación por categorías para una experiencia de usuario fluida.
- **Juegos:**
  - Minijuego de memoria incorporado en la aplicación en el que el usuario debe encontrar las parejas de cartas de aldeanos.
  - Incluye 5 niveles de dificultad (entre 9 y 25 parejas) y una fase inicial de memorización. El sistema guarda la mejor marca de cada usuario en cada nivel (tiempo y número de movimientos) y solo la actualiza cuando se mejora el récord previo.

### 2.3 Diagramas de casos de uso

En este apartado se describen las funcionalidades del sistema desde la perspectiva de los actores que interactúan con la plataforma "Canela's Desk".

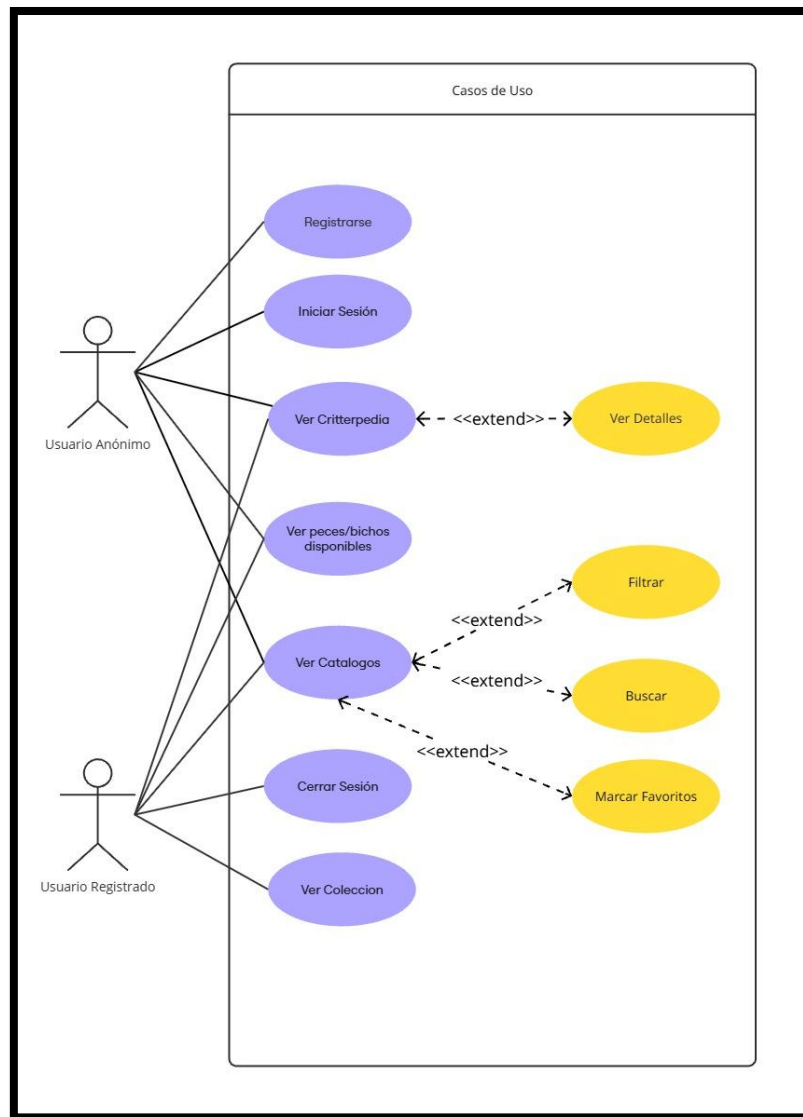


Figura 1 Diagrama Casos de Uso.

### 2.3.1. Identificación de Actores

- **Usuario Anónimo (Visitante):** Usuario que accede a la plataforma sin autenticarse. Tiene permisos de consulta básica.
- **Usuario Registrado:** Usuario autenticado que, además de las funciones de consulta, posee persistencia de datos y una colección personal.

### 2.3.2. Descripción de los Casos de Uso

A continuación, se detallan las interacciones representadas en el diagrama:

#### Gestión de Sesión y Acceso

- **Registrarse / Iniciar Sesión:** El usuario anónimo puede crear una cuenta o acceder a una existente para desbloquear funciones personalizadas.



- **Cerrar Sesión:** Acción exclusiva del usuario registrado para finalizar su sesión de forma segura.
- **Consulta de Información (Core de la App)**

#### Ver Critterpedia:

Tabla 1. Caso de uso de Critterpedia.

| Caso de Uso                   | CU-03: Ver Critterpedia                                                                                                                                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actores                       | Usuario Anónimo, Usuario Registrado                                                                                                                                                          |
| Descripción                   | El usuario accede al listado general de bichos, peces y criaturas marinas.                                                                                                                   |
| Relaciones                    | Extiende a: CU-04 Ver Detalles.                                                                                                                                                              |
| Flujo Principal               | El usuario selecciona la sección "Critterpedia" en el menú.<br>El sistema realiza una petición al endpoint de la API.<br>3. Se renderiza un grid con las tarjetas básicas de los ejemplares. |
| Flujo Alternativo (Extensión) | E1. Ver Detalles: Si el usuario hace clic en una tarjeta, el sistema activa el componente CritterDetail para mostrar información técnica avanzada (precios, horarios, frases).               |

Ver peces/bichos disponibles: Acceso directo a la funcionalidad de filtrado por tiempo real (estacionalidad).

#### Ver Catálogos:

Tabla 2. Caso de uso de Ver Catálogos.

| Caso de Uso                   | CU-05: Ver Catálogos                                                                                                                                                                                                                                                                                   |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actores                       | Usuario Anónimo, Usuario Registrado                                                                                                                                                                                                                                                                    |
| Descripción                   | Permite navegar por las colecciones de Bichos, Peces, Criaturas Marinas, Fósiles, Arte y Vecinos.                                                                                                                                                                                                      |
| Relaciones                    | Extiende a: Filtrar, Buscar y Marcar Favoritos.                                                                                                                                                                                                                                                        |
| Flujo Principal               | El usuario elige una categoría (ej. Vecinos).<br>El sistema carga los datos correspondientes desde la BD.<br>3. El usuario visualiza la lista de elementos.                                                                                                                                            |
| Flujo Alternativo (Extensión) | E1-2. Buscar/Filtrar: El sistema integra herramientas de búsqueda por nombre y filtros por atributos (personalidad, especie, etc.) para segmentar la vista.<br>E3. Marcar Favoritos: Si el usuario está registrado, puede pulsar el icono de "corazón" para añadir el elemento a su colección privada. |

Marcar Favoritos: Es una extensión del catálogo. Solo si el usuario está interesado (y registrado), puede marcar un elemento para su seguimiento personal.

## Gestión Personalizada

Ver Colección: Funcionalidad clave para el usuario registrado, donde se visualiza el progreso de su museo personal y sus residentes favoritos.

## 3 Diseño

### 3.1 Modelo de base de datos

#### 3.1.1 Diagrama Entidad-Relación:

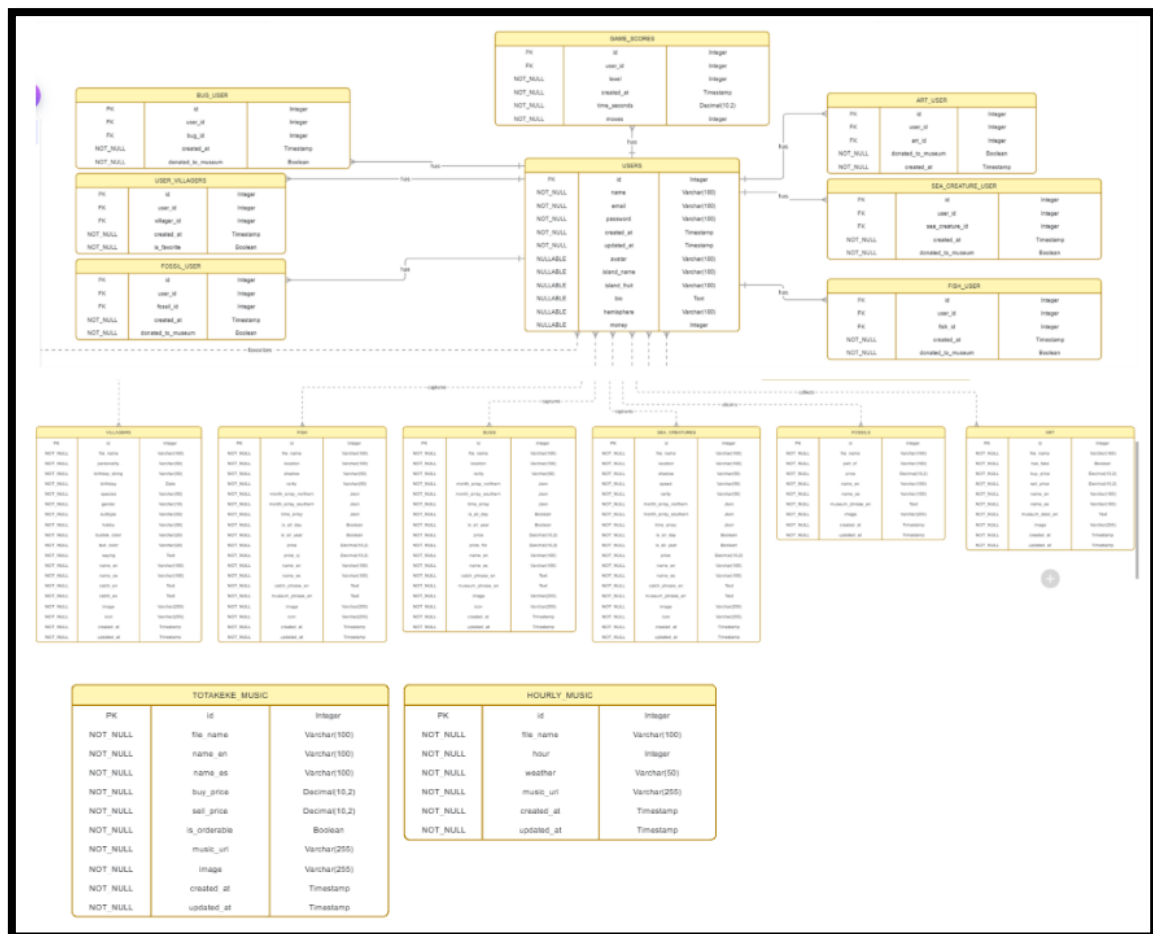


Figura 2. Diagrama Entidad-Relación.

### Justificación del modelo entidad-relación

El modelo entidad-relación propuesta se diseñó con el objetivo de representar de forma estructurada y normalizada la información del sistema, evitando redundancias y garantizando la integridad de los datos. Para ello, se identificaron las entidades principales del dominio y se definieron correctamente sus relaciones.

### Tabla Usuarios

La tabla usuarios almacena la información básica de cada usuario del sistema, como su identificador único, nombre, correo y otros datos relevantes. Esta entidad actúa como eje central del modelo, ya que un usuario puede interactuar con múltiples elementos del sistema, como favoritos, objetos o personajes.

Relaciones:

- Un usuario puede tener varios favoritos (relación 1-N).
- Un usuario puede estar asociado a múltiples elementos del juego mediante tablas intermedias.

### **Tabla Favoritos**

La tabla favoritos se utiliza para registrar los elementos que un usuario marca como preferidos.

Funciona como una tabla intermedia, resolviendo una relación muchos-a-muchos entre usuarios y los elementos que pueden ser favoritos.

Relaciones:

- Cada registro pertenece a un solo usuario.
- Cada registro referencia un solo elemento, pero un usuario puede tener muchos favoritos y un elemento puede ser favorito de muchos usuarios.
- Esta tabla evita la duplicación de datos y permite flexibilidad en la gestión de preferencias.

### **Tablas de elementos principales (por ejemplo: peces, aldeanos, fósiles, etc.)**

Las tablas que representan los distintos elementos del sistema (aldeanos, peces u otros objetos) almacenan información específica de cada tipo de entidad, como nombre, características y atributos propios.

Relaciones:

- Estas entidades pueden relacionarse con usuarios de forma indirecta a través de tablas intermedias.
- Permiten reutilizar la información sin necesidad de repetirla para cada usuario.

### **Tablas intermedias**

Las tablas intermedias se utilizan para modelar relaciones muchos-a-muchos, lo cual mejora la normalización del modelo. Gracias a ellas, se pueden añadir atributos adicionales a la relación (por ejemplo, fecha, estado o cantidad) sin modificar las entidades principales.

## **Normalización e integridad**

El modelo sigue principios de normalización:

- Cada tabla tiene una clave primaria que identifica de forma única cada registro.
- Las claves foráneas aseguran la integridad referencial entre las tablas.
- Se evita la redundancia de información y se facilita el mantenimiento del sistema.

## Conclusión

En conjunto, el modelo entidad-relación permite una gestión clara, escalable y coherente de los datos. La separación en entidades bien definidas y el uso de relaciones adecuadas facilitan la ampliación del sistema y garantizan un acceso eficiente a la información.

## 3.2 Diseño de la interfaz

### 3.2.1 Prototipo

Para el desarrollo del prototipo, se ha seguido un flujo de trabajo que integra inteligencia artificial y diseño responsivo, asegurando un resultado profesional en tiempo récord.

#### 1. Herramientas Utilizadas

- **Figma + Figma AI (Maker):** Se utilizó como entorno principal para el diseño y la ideación. Las herramientas de generación de Figma Maker permitieron agilizar la creación de la estructura base y los componentes visuales iniciales.
- **Lucide React:** Para la implementación de iconografía ligera y moderna.
- **Tailwind CSS:** Para garantizar que el diseño sea totalmente adaptativo.

El diseño de la interfaz de Canela's Desk se ha desarrollado bajo un enfoque de prototipado de alta fidelidad, buscando definir no solo la estética, sino la usabilidad completa antes de la fase de implementación.

#### 2. Estrategia de Diseño Responsivo (Desktop & Mobile)

- **Versión Desktop:** Diseñada para una interacción de escritorio, donde prima la organización de la información y la amplitud visual, permitiendo al usuario gestionar sus colecciones con comodidad.
- **Versión Mobile:** Se ha priorizado la ergonomía táctil. En esta versión, el diseño se adapta para que los elementos interactivos sean fáciles de accionar con una sola mano, ajustando jerarquías visuales para evitar la fatiga cognitiva en pantallas pequeñas.

#### 3. Metodología de Trabajo

El proceso no fue lineal, sino que se dividió en tres fases críticas:

- **Generación y Estructura:** Con Figma Maker se establecieron los marcos (frames) y la jerarquía de contenidos de "Canela's Desk", permitiendo pasar de una idea conceptual a un prototipo visual en minutos.
- **Refinamiento Responsivo:** Tras la generación inicial, se realizó un ajuste manual para asegurar que tanto la versión Desktop como la Mobile fueran coherentes. Se ajustaron márgenes y la disposición de los elementos para optimizar la usabilidad.
- **Implementación de Estados:** Se definieron micro-interacciones para validar la lógica funcional antes del desarrollo final.

#### 4. Publicación

El prototipo ha sido publicado de forma independiente para facilitar su revisión en dispositivos reales. Se puede acceder a través de los siguientes enlaces:

Prototipo Publicado: <https://buck-buffet-56277342.figma.site>

#### Diseño mobile



Figura 3. Adaptación responsiva para dispositivos móviles. El contenido se reorganiza en una columna única (stack) y se ajusta el navbar

### Diseño Desktop

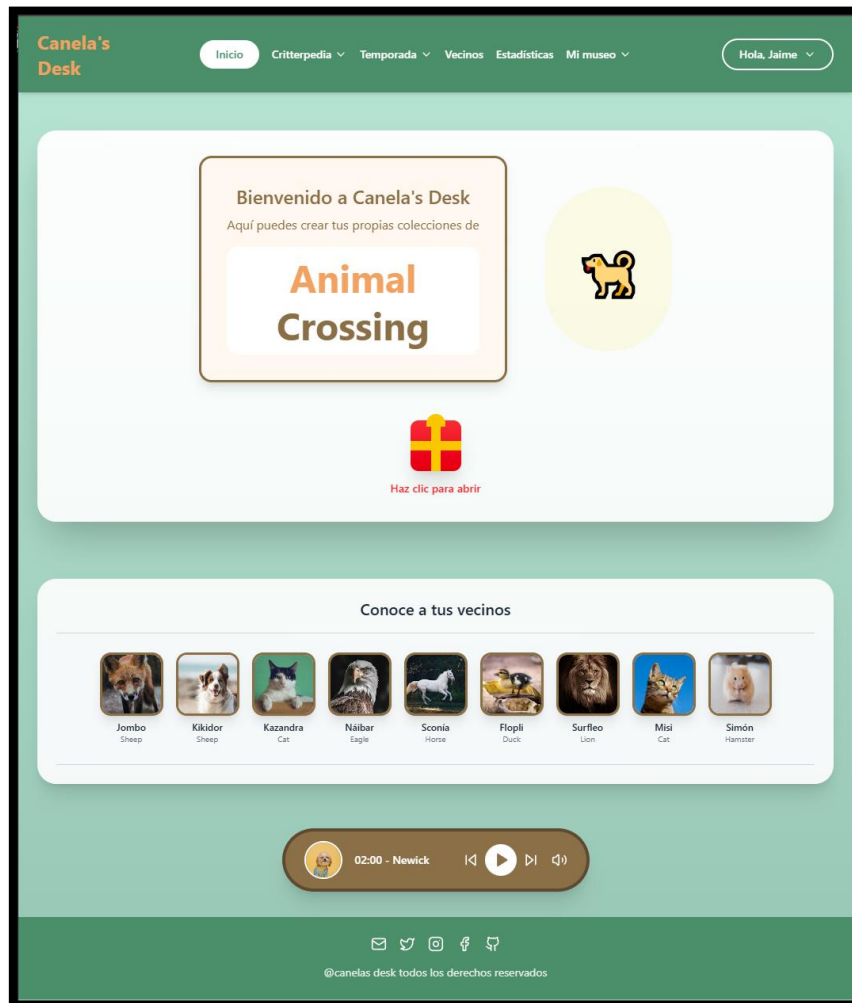


Figura 4. Vista general del prototipo en versión Desktop. Se observa la disposición horizontal de los elementos y el uso de la tipografía Nunito.

### 3.2.2 Guía de estilos

#### 1. Estrategia Cromática

La paleta de colores se ha definido mediante una auditoría técnica con Adobe Color, extrayendo los valores directamente del logotipo para garantizar la coherencia de marca. Se divide en dos ejes:

- **Colores de Identidad (Logo-based):** Uso de dorados (#F2B035) y marrones (#9A722C) para elementos de acción y jerarquía, heredando la tridimensionalidad y cercanía del logo.
- **Colores Atmosféricos:** Integración de verdes y azules pastel para evocar naturaleza y frescura, junto con fondos crema (#faf8ef) que reducen la fatiga visual.

- **Legibilidad:** Se ha sustituido el negro puro por un marrón oscuro (#3b3022) para el texto, manteniendo un alto contraste, pero con una estética más suave y orgánica.

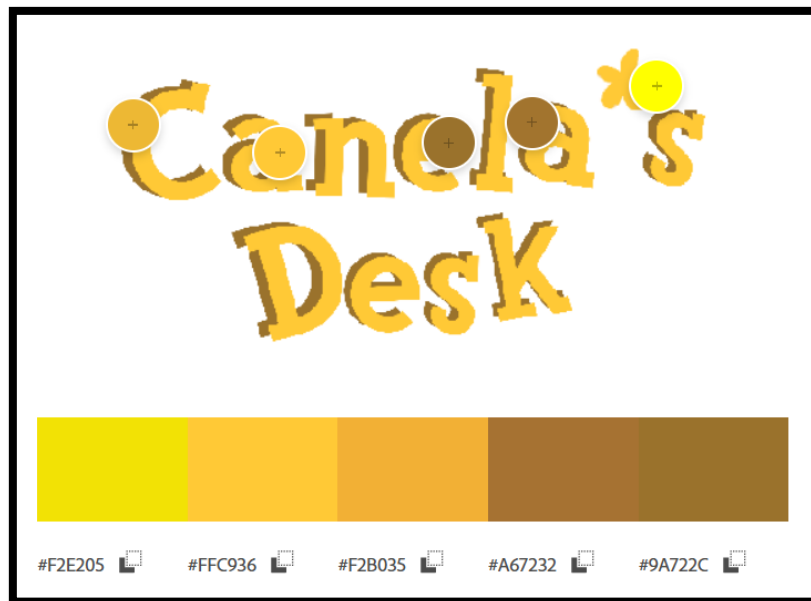


Figura 5. Colores de Identidad (Extraídos con Adobe Color)

## 2. Tipografía

Se han seleccionado fuentes que refuerzan el tono amigable del proyecto:

- **Nunito (Principal):** Fuente redondeada que aporta cercanía y una excelente legibilidad en dispositivos móviles.
- **Figtree (Soporte):** Utilizada para elementos funcionales de la interfaz, aportando un equilibrio moderno y limpio.

# 4 Desarrollo

## 4.1 Stack tecnológico

### Backend:

- Laravel 12 (PHP 8.3+)
- MySQL 8.4 como sistema gestor de base de datos
- Composer para gestión de dependencias PHP

### Frontend:

- Vue.js 3.5 con Composition API (<script setup>)
- Inertia.js para comunicación seamless frontend-backend
- Tailwind CSS 3.4 y 4.1 para estilos utility-first
- Axios para peticiones HTTP asíncronas

- Vite como build tool y servidor de desarrollo

#### *Herramientas de desarrollo:*

- Node.js 22 y npm para gestión de paquetes JavaScript
- Git para control de versiones
- Postman para testing de API

#### *Software:*

- Visual Studio Code con extensiones: Vue, Laravel Extension Pack, Tailwind CSS IntelliSense
- HeidiSQL para diseño y gestión de base de datos
- Laragon como servidor local de desarrollo
- Chrome DevTools para análisis de rendimiento y responsive design

## 4.2 Estructura de carpetas

La arquitectura de Canela's Desk sigue el patrón de desarrollo de una aplicación monolítica moderna, integrando el motor de Laravel con Inertia.js y Vue.js para una experiencia de usuario fluida sin recargas de página.

### 4.2.1 Estructura de Directorios

canelas-desk/

└─ Backend/

└─ app/           # Lógica de negocio (Controladores y Modelos)

└─ database/       # Migraciones y Seeders (Estructura de BD)

└─ public/         # Recursos estáticos (Imágenes, música, iconos)

└─ resources/

| └─ css/         # Estilos globales y específicos (Tailwind CSS)

| └─ js/          # Frontend: Páginas y Componentes en Vue.js

└─ routes/        # Definición de rutas (Web, API y Auth)

### 4.2.2 Descripción de Directorios Clave

1.- **Backend/**: Directorio raíz que contiene la lógica integral del proyecto. Al utilizar Inertia.js, el backend y el frontend conviven en el mismo ecosistema, facilitando el despliegue y la comunicación de datos.

2.- **database/**: Contiene las definiciones de la base de datos (migraciones) y los seeders utilizados para poblar la aplicación con datos de prueba (catálogo de objetos de Animal Crossing).



3.-**public/**: Almacena los activos accesibles públicamente, incluyendo la biblioteca de sonidos/música ambiente y la iconografía del proyecto.

4.- **resources/js/**: Es el núcleo del frontend. Aquí se gestionan los estados de Vue y la lógica interactiva de las colecciones.

5.-**resources/css/**: Centraliza el uso de Tailwind CSS, permitiendo que la paleta de colores definida (extraída mediante Adobe Color) se aplique de forma consistente en toda la interfaz.

6.-**routes/**: Gestiona tanto la navegación de la página como los sistemas de autenticación de usuarios.

### 4.3 Arquitectura del sistema

La aplicación está construida bajo el patrón MVC (Modelo-Vista-Controlador), enriquecido con una Capa de Servicios (Service Layer). En esta estructura, los controladores actúan únicamente como coordinadores, delegando la lógica de negocio a servicios especializados.

#### 4.3.1 Ventajas de este enfoque:

- Controladores "Lean" (ligeros): Se limitan a gestionar la entrada y salida de datos.
- Reutilización: La lógica reside en servicios que pueden ser invocados desde múltiples controladores o comandos de consola.
- Testabilidad: Facilita la implementación de pruebas unitarias sobre la lógica de negocio de forma aislada.

#### 4.3.2 Flujo de Datos

1. Request HTTP: La petición entra al sistema.
2. Routes: Se dirige a web.php o api.php.
3. Middleware: Verificación de seguridad y estado (Auth, Sanctum).
4. Controller: Recibe la petición y orquesta la respuesta.
5. Service: Ejecuta la lógica compleja y gestiona la caché.
6. Model: Interactúa con la base de datos mediante Eloquent.
7. Resource: Transforma los modelos en estructuras JSON estandarizadas.
8. Response: Envío de la respuesta al cliente.

(Inertia.js: Para las vistas de la plataforma, el controlador utiliza `Inertia::render()`. Esto permite enviar los datos directamente a los componentes Vue, eliminando la necesidad de peticiones AJAX adicionales en la carga inicial y ofreciendo una experiencia de Single Page Application (SPA))

#### 4.3.3 Capa de Datos (Models)

Se utiliza Eloquent ORM para representar las tablas y gestionar las relaciones.

### *Abstracción de Lógica con Traits*

Para evitar la duplicidad de código en coleccionables similares (Peces, Bichos, Criaturas Marinas), se ha implementado el trait HasAvailability. Este encapsula la lógica de filtrado por tiempo y hemisferio.

```
// app/Traits/HasAvailability.php
trait HasAvailability
{
    public function scopeAvailableNow(Builder $query, string $hemisphere = 'north')
    {
        $month = now()->month;
        $hour = now()->hour;

        $monthField = $hemisphere === 'south'
            ? 'month_array_southern'
            : 'month_array_northern';

        return $query
            ->where(function ($q) use ($month, $monthField) {
                $q->whereJsonContains($monthField, $month)
                ->orWhere('is_all_year', true);
            })
            ->where(function ($q) use ($hour) {
                $q->whereJsonContains('time_array', $hour)
                ->orWhere('is_all_day', true);
            });
    }
}
```

Figura 6. Modelo Fish con trait reutilizable

### *Gestión de Colecciones (Relaciones Pivot)*

La relación entre usuarios y coleccionables se gestiona mediante tablas pivot, permitiendo trackear el estado de donación de forma individual.

```
// app/Models/User.php (fragmento)
public function fish()
{
    return $this->belongsToMany(Fish::class, 'fish_user')
        ->withPivot('donated_to_museum')
        ->withTimestamps();
}

public function bugs()
{
    return $this->belongsToMany(Bug::class, 'bug_user')
        ->withPivot('donated_to_museum')
        ->withTimestamps();
}
```

Figura 7. El modelo User define relaciones muchos-a-muchos con cada tipo de coleccionable, usando tablas pivot que almacenan

### Capa de Servicios (Service Layer)

Los servicios centralizan la lógica operativa del sistema. A continuación, se detallan las responsabilidades principales:

- CatalogService: Gestión de catálogos, aplicación de filtros dinámicos y almacenamiento en caché.
- CollectionService: Lógica de "toggle" para donaciones y obtención de ítems del usuario.
- MuseumStatsService: Cálculo de progreso del museo con invalidación de caché inteligente.
- MemoryGameService: Lógica del minijuego: generación de mazos y persistencia de récords.
- ReCaptchaService: Validación de seguridad contra bots mediante API externa.

### Capa de Controladores

Gracias a la inyección de servicios, los controladores son extremadamente compactos (generalmente menos de 30 líneas), facilitando el mantenimiento.

```
// app/Http/Controllers/FishController.php
class FishController extends Controller
{
    public function __construct(
        private readonly CatalogService $catalogService,
    ) {}

    public function index()
    {
        return FishResource::collection($this->catalogService->getAll(Fish::class));
    }

    public function filter(Request $request)
    {
        $query = $this->catalogService->applyFilters(Fish::class, $request, ['rarity', 'location']);
        return FishResource::collection($query->get());
    }

    public function available(Request $request)
    {
        $hemisphere = $request->get('hemisphere', 'north');
        return FishResource::collection($this->catalogService->getAvailable(Fish::class, $hemisphere));
    }
}
```

Figura 8. cada controlador tiene, menos de 30 líneas

**Optimización en DonationController:** En lugar de crear un controlador por cada tipo de ítem, se implementó un RELATIONSHIP\_MAP. Esto permite gestionar donaciones de peces, fósiles o arte mediante una única lógica centralizada, cumpliendo el principio DRY (Don't Repeat Yourself).

```
DonationController: Consolida 5 controladores en uno usando un mapa de relaciones.

// app/Http/Controllers/DonationController.php
class DonationController extends Controller
{
    private const RELATIONSHIP_MAP = [
        'fish'      => 'fish',
        'bugs'      => 'bugs',
        'fossils'   => 'fossils',
        'art'       => 'art',
        'sea_creatures' => 'seaCreatures',
    ];

    public function __construct(
        private readonly CollectionService $collectionService,
        private readonly MuseumStatsService $museumStatsService,
    ) {}

    public function donate(Request $request, string $type, int $itemId): JsonResponse
    {
        $relationship = self::RELATIONSHIP_MAP[$type];
        $user = $request->user();

        $this->collectionService->toggle($user, $itemId, $relationship);
        $this->museumStatsService->invalidateCache($user->id);

        return response()->json(['success' => true]);
    }
}
```

Figura 9. Consolida 5 controladores en uno usando un mapa de relaciones

### Frontend: Componibilidad y Estado

En el lado del cliente (Vue 3), se ha optado por la Composition API para crear una lógica reactiva y limpia.

### Gestión de Endpoints

Para garantizar la mantenibilidad, se ha implementado un diccionario centralizado de rutas API. Esto evita el uso de "Magic Strings" (URLs escritas a mano) en los componentes, facilitando cambios globales en la estructura de las rutas desde un único punto de verdad.

```
// resources/js/Uutils/api.js
export const API = {
  bugs: {
    list: 'api/bugs',
    filter: 'api/bugs/filter',
    available: 'api/bugs/available',
  },
  fish: {
    list: 'api/fish',
    filter: 'api/fish/filter',
    available: 'api/fish/available',
  },
  // ... otros catalogos
  user: {
    items: (type) => `user/${type}`,
  },
  donate: (type, id) => `${type}/${id}/donate`,
  favorite: (id) => `villagers/${id}/favorite`,
}
```

Figura 10. Todos los endpoints de la API están centralizados en un único fichero para facilitar el mantenimiento.

### Composables Destacados

- **useFetch:** Un wrapper sobre Axios que gestiona automáticamente los estados de loading, error y data, soportando reactividad en las URLs.

```
// resources/js/Composables/useFetch.js
export function useFetch(url, options = { debounce: 0 }) {
  const data = ref(null)
  const loading = ref(true)
  const error = ref(null)

  const fetchData = async () => {
    const urlValue = isRef(url) ? url.value : url
    if (!urlValue) return

    loading.value = true
    try {
      const res = await axios.get(urlValue)
      data.value = res.data
    } catch (e) {
      error.value = e
    } finally {
      loading.value = false
    }
  }

  if (isRef(url)) {
    watch(url, () => fetchData())
  }

  fetchData()
  return { data, loading, error }
}
```

Figura 11. useFetch: Peticiones GET reactivas con estado de carga.

- **useMuseum:** Gestiona el estado de las donaciones en tiempo real. Implementa Actualizaciones Optimistas: el UI se actualiza instantáneamente antes de recibir la confirmación del servidor, revirtiendo el estado solo en caso de error. Esto mejora drásticamente la percepción de velocidad del usuario.

```
// resources/js/Composables/useMuseum.js
export function useMuseum(type = 'bugs') {
  const { data: userData } = useFetch(API.user.items(type))
  const museumIds = ref(new Set())

  watch(userData, (newData) => {
    if (Array.isArray(newData)) {
      museumIds.value = new Set(newData.map(item => Number(item.id)))
    }
  }, { immediate: true })

  const isInMuseum = (id) => museumIds.value.has(Number(id))

  const toggleDonation = async (id) => {
    const numericId = Number(id)

    // Actualización optimista: cambia el UI inmediatamente
    if (museumIds.value.has(numericId)) {
      museumIds.value.delete(numericId)
    } else {
      museumIds.value.add(numericId)
    }
    museumIds.value = new Set(museumIds.value)

    try {
      await axios.post(API.donate(type, numericId))
    } catch (err) {
      // Revertir si falla
      if (museumIds.value.has(numericId)) museumIds.value.delete(numericId)
      else museumIds.value.add(numericId)
      museumIds.value = new Set(museumIds.value)
    }
  }

  return { museumIds, isInMuseum, toggleDonation }
}
```

Figura 12. useMuseum: Gestiona el estado de donaciones con actualización optimista.

### UI basada en Atomic Design

Se han desarrollado componentes base (como AppButton.vue) que centralizan el diseño y comportamiento de la interfaz, permitiendo cambios globales de estilo con un esfuerzo mínimo.

```

<!-- resources/js/Components/Base/AppButton.vue -->
<script setup>
const props = defineProps({
  variant: {
    type: String,
    default: 'primary',
    validator: (v) => ['primary', 'secondary', 'danger', 'ghost'].includes(v),
  },
  disabled: { type: Boolean, default: false },
  loading: { type: Boolean, default: false },
})

const variantClasses = computed(() => ({
  primary: 'btn-primary',
  secondary: 'btn-secondary',
  danger: 'ac-btn-danger',
  ghost: 'bg-transparent hover:bg-gray-100 text-gray-700',
})[props.variant])
</script>

<template>
  <button
    :class="variantClasses"
    :disabled="disabled || loading"
    :aria-busy="loading"
  >
    <span v-if="loading" class="animate-spin mr-2"></span>
    <slot />
  </button>
</template>

```

Figura 13. AppButton: Botón con variantes y estado de carga

### Seguridad y Validación

- Validación Robusta: Se utilizan FormRequests para desacoplar las reglas de validación de los controladores.

```

// app/Http/Requests/GameScoreRequest.php
class GameScoreRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'level' => 'required|integer|between:1,5',
            'time_seconds' => 'required|numeric|min:0.001',
            'moves' => 'required|integer|min:1',
        ];
    }
}

```

Figura 14. Los FormRequests extraen la lógica de validación de los controladores, haciéndola reutilizable y testeable.

- Protección de Datos: Las contraseñas se gestionan mediante el algoritmo de hashing Bcrypt.

```
// app/Http/Controllers/Auth/RegisteredUserController.php
$user = User::create([
    'name'      => $request->name,
    'email'     => $request->email,
    'password' => Hash::make($request->password),
]);
```

Figura 15. Encriptación de contraseñas: Se utiliza Bcrypt mediante Hash::make().

- Seguridad en Formularios: Integración de Google ReCaptcha v2/v3 para prevenir ataques de fuerza bruta en el registro y login.

```
// app/Services/ReCaptchaService.php
class ReCaptchaService
{
    public function verify(string $token, ?string $ip = null): void
    {
        $response = Http::asForm()->post(config('recaptcha.verify_url'), [
            'secret' => config('recaptcha.secret_key'),
            'response' => $token,
            'remoteip' => $ip,
        ]);

        if (!$response->json('success')) {
            throw ValidationException::withMessages([
                'captcha_token' => 'La verificación del captcha ha fallado.',
            ]);
        }
    }
}
```

Figura 16. Verificación ReCaptcha: Un servicio dedicado valida el token antes de procesar login o registro.

## 4.4 Descripción del funcionamiento

### 4.4.1 Manual de usuario

#### Navegación inicial:

Al acceder a la aplicación, el usuario visualiza la página principal, donde se muestra:

- Un mensaje de bienvenida.
- Un regalo diario que indica el personaje que cumple años ese día.
- Una pequeña galería de personajes.
- Un reproductor de música con la banda sonora del juego, la cual varía según la hora del día.





Figura 17. Pantalla principal.

En la parte superior se encuentra el menú de navegación, desde el cual el usuario puede acceder a las siguientes secciones:

- Inicio
- Critterpedia
- Temporada
- Vecinos

Además, junto a la barra de navegación aparece el botón “Identifícate”, que al pulsarlo despliega las opciones de Login y Registro.

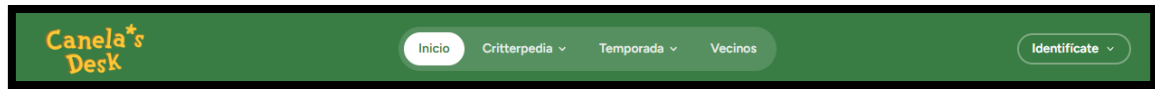


Figura 18. Barra de navegación para usuarios anónimos.

### Usuario logueado

Una vez el usuario inicia sesión, el menú de navegación se amplía y aparecen nuevas secciones:

- Estadísticas
- Mi museo

Asimismo, el botón “Identificate” cambia su contenido y permite acceder a las opciones de Perfil y Cerrar sesión.



Figura 19. Barra de navegación para usuarios registrados.

### Exploración de las pestañas

Al pasar por las distintas pestañas del menú de navegación, se muestran desplegables específicos según la categoría seleccionada. Al pulsar sobre cualquiera de estas opciones, el contenido de la página se actualiza de forma dinámica, sin necesidad de recargar la aplicación.

### Criterpedia

Al acceder a la Criterpedia y seleccionar cualquiera de sus categorías:

- En la parte izquierda aparece una lista de iconos.
- Al seleccionar uno de ellos, se muestra a la derecha una tarjeta (card) con la información detallada del elemento seleccionado.

Si el usuario se encuentra logueado, se habilita un botón con un icono de museo, que permite donar el objeto seleccionado al museo personal.

En la parte superior se incluye una barra de búsqueda junto con diferentes botones de filtrado, que facilitan la localización de la información.

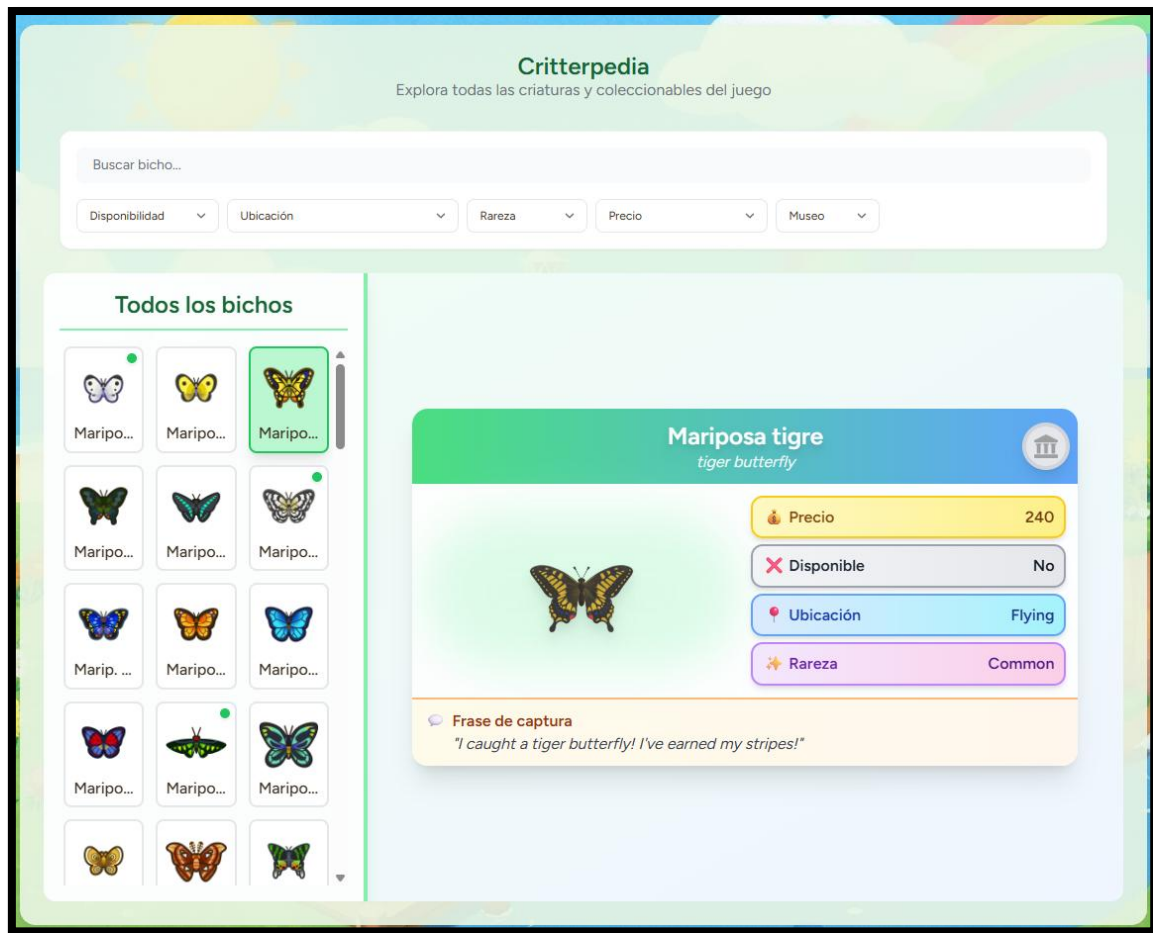


Figura 20. Pestaña Critterpedia. Aquí se observa la información detallada de los bichos, junto con un botón para donar al museo (dolo para usuarios logueados).

### Temporada

En la sección Temporada se muestran diferentes tarjetas con la fauna capturable durante el mes actual, indicando:

- Ubicación
- Horario de disponibilidad

En la parte superior se dispone de una barra con tres categorías, que permite filtrar el contenido mostrado.

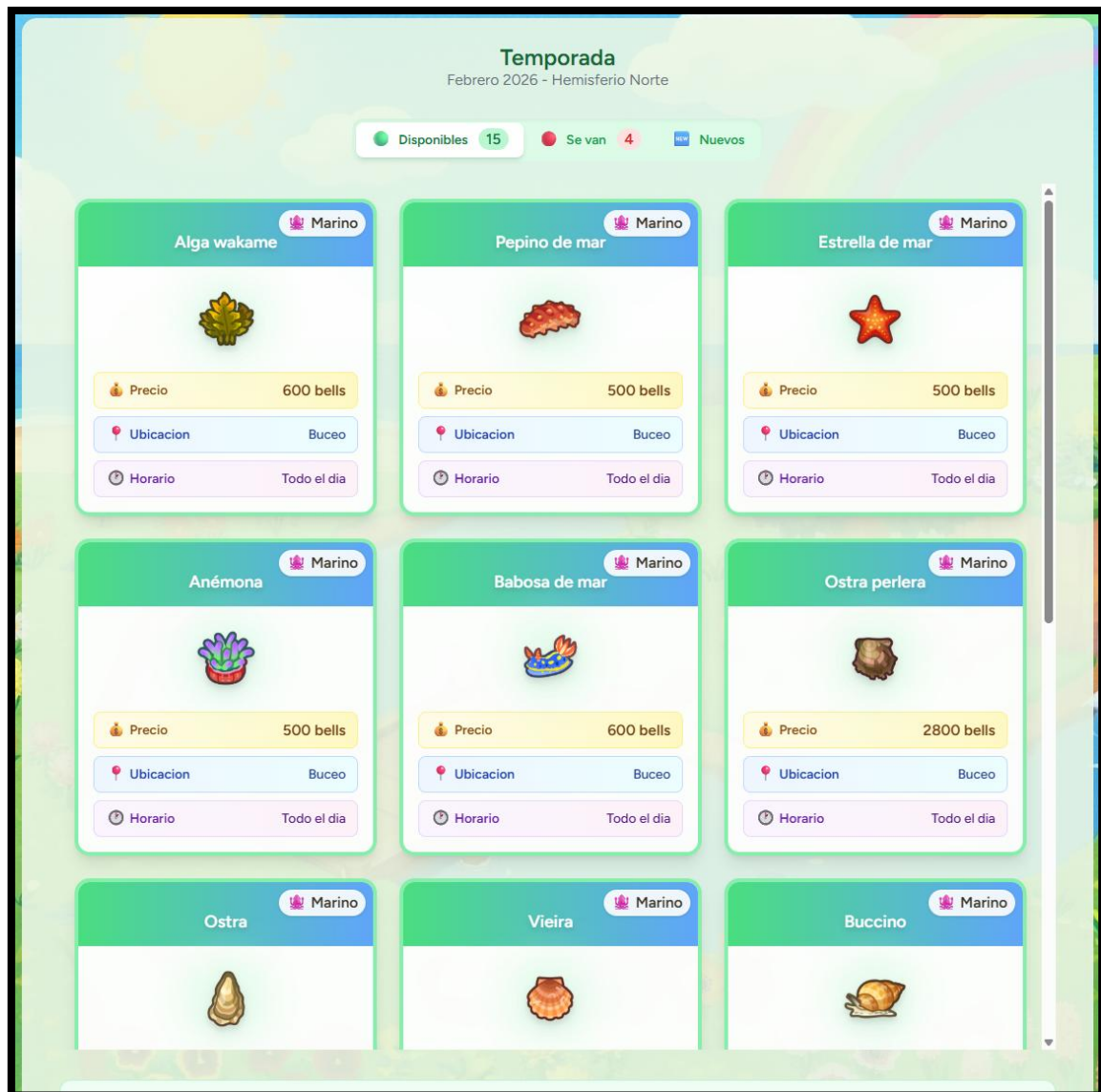


Figura 21. Pestaña de Temporada. Aquí se muestra la disponibilidad de los items, en este caso criaturas marinas.

## Vecinos

Al acceder a Vecinos, se presentan distintas tarjetas de los aldeanos. Al pulsar sobre una de ellas, se despliega toda su información detallada.

Si el usuario está logueado, aparece un icono de corazón en la esquina superior derecha de cada tarjeta, que permite añadir al vecino a la lista de favoritos.

Además, la sección cuenta con:

- Una barra de búsqueda con filtros en la parte superior.
- Una barra de paginación en la parte inferior para navegar entre las diferentes páginas de vecinos.

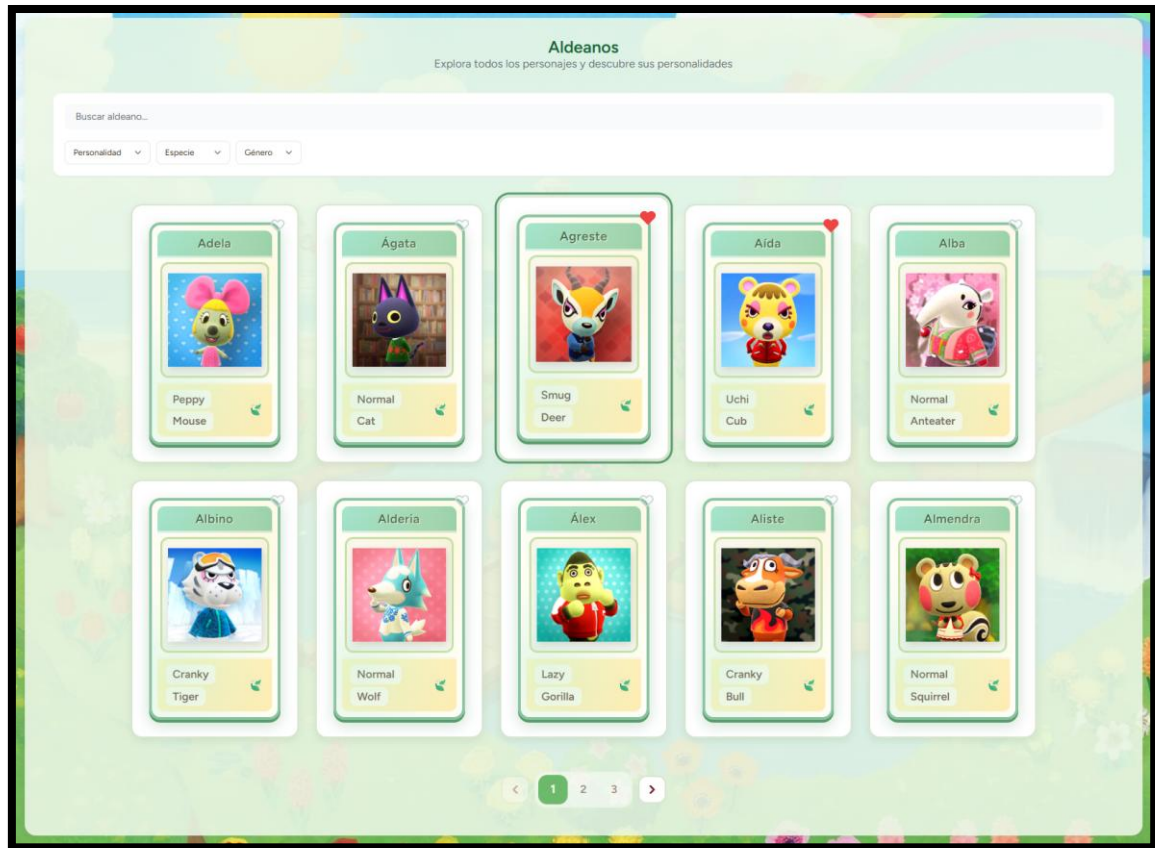


Figura 22. Pestaña de Vecinos. Aquí se observan los aldeanos del juego, los cuales puedes agregar a favoritos (solo para usuarios logueados).

### Estadísticas

En la pestaña Estadísticas se muestran distintos gráficos que representan:

- El progreso total del museo.
- El progreso individual por cada categoría.

También se incluyen varias tarjetas informativas con datos detallados del avance por categoría.



Figura 23. Pestaña de Estadísticas. Aquí se muestra el progreso de las colecciones del usuario

### Mi museo

En la sección Mi museo, el usuario puede consultar:

- Los objetos donados al museo, organizados por categorías.
- Los vecinos marcados como favoritos.



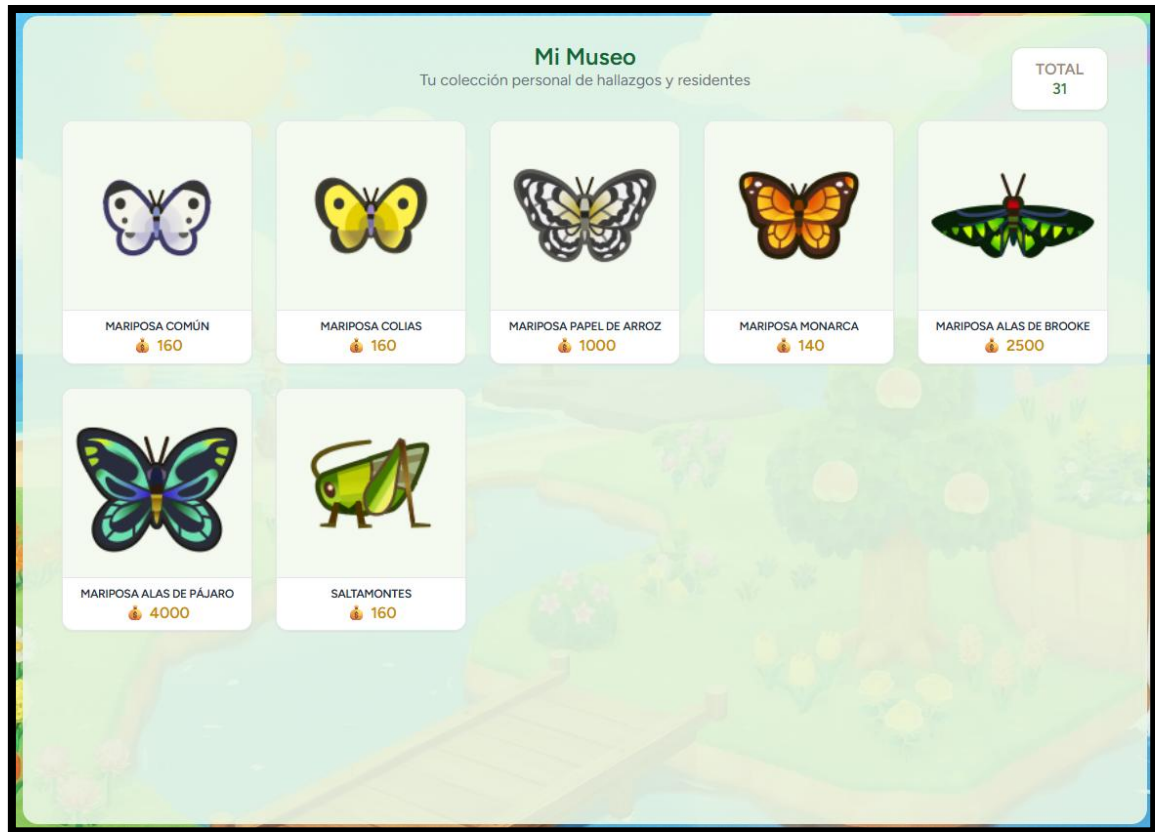


Figura 24. Pestaña de Museo. Aquí se guarda la colección del usuario.

### Login

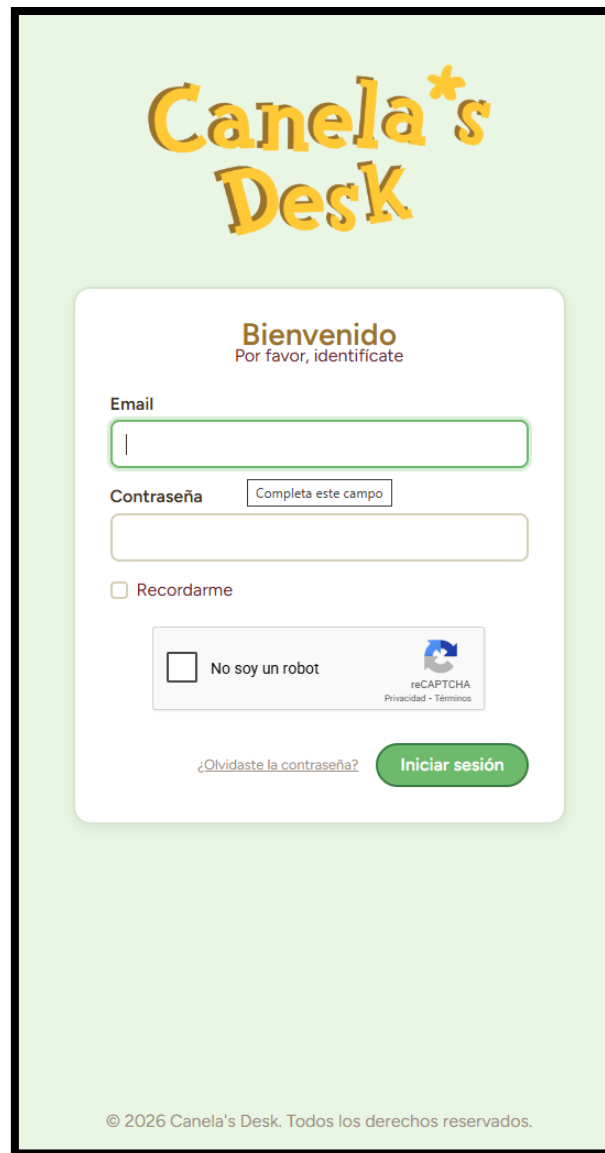
Al pulsar en Login, se muestra un formulario de acceso donde el usuario debe introducir:

- Correo electrónico
- Contraseña

Además, se incluyen las opciones para:

- Recordar los datos de acceso.
- Cambiar la contraseña.

Para completar el inicio de sesión, es necesario resolver un captcha de verificación.



The image shows a login screen for 'Canela's Desk'. At the top, the logo 'Canela's Desk' is displayed in a yellow, stylized font. Below the logo, the text 'Bienvenido' (Welcome) is shown in bold, followed by 'Por favor, identificate' (Please identify yourself). The form contains two input fields: 'Email' and 'Contraseña' (Password). The 'Contraseña' field has a placeholder text 'Completa este campo' (Complete this field). Below the password field is a checkbox labeled 'Recordarme' (Remember me). A reCAPTCHA widget is present with the text 'No soy un robot' (I am not a robot) and a small robot icon. To the right of the reCAPTCHA widget are links for 'Privacidad' (Privacy) and 'Términos' (Terms). At the bottom left, there is a link '¿Olvidaste la contraseña?' (Forgot your password?). At the bottom right, there is a green button labeled 'Iniciar sesión' (Log in). The footer text at the bottom of the screen reads '© 2026 Canela's Desk. Todos los derechos reservados.' (© 2026 Canela's Desk. All rights reserved.).

Figura 25. Pantalla de Inicio de Sesión.

### Registro

En la sección Registro se presenta un formulario con los datos necesarios para crear una nueva cuenta. También se incluye un botón que permite acceder directamente al Login en caso de que el usuario ya disponga de una cuenta.

Para completar el registro, es necesario resolver un captcha de verificación.



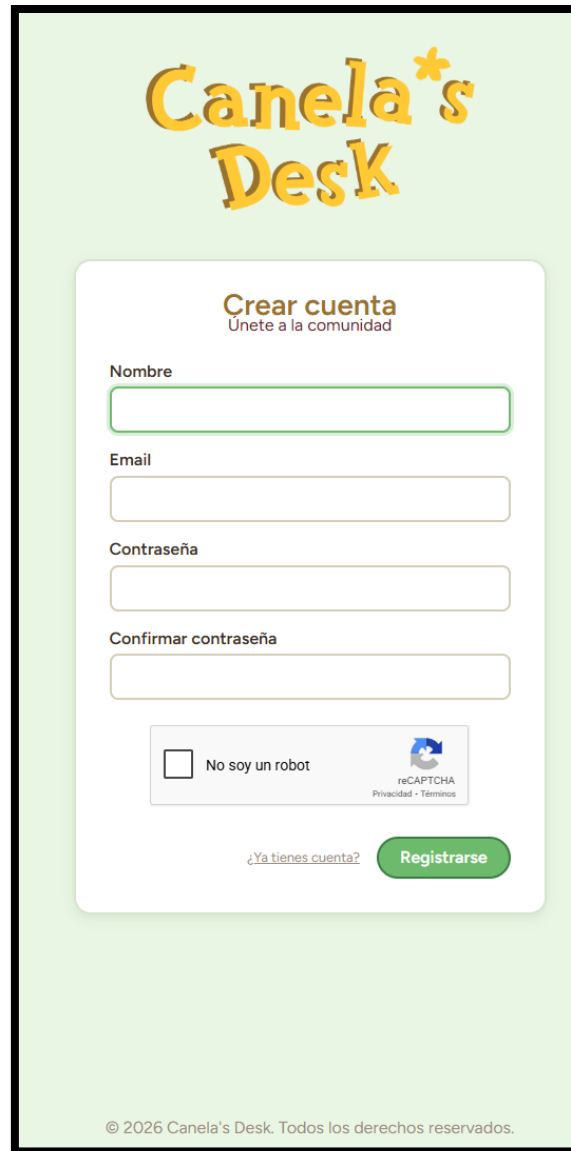
The image shows a registration form for 'Canela's Desk'. At the top, the logo 'Canela's Desk' is displayed in a yellow, stylized font. Below the logo, the heading 'Crear cuenta' (Create account) is shown in bold, with the subtitle 'Únete a la comunidad' (Join the community) underneath. The form contains four input fields: 'Nombre' (Name), 'Email', 'Contraseña' (Password), and 'Confirmar contraseña' (Confirm password). Below these fields is a checkbox labeled 'No soy un robot' (I am not a robot) next to a reCAPTCHA logo and the text 'reCAPTCHA Privacidad · Términos'. At the bottom of the form, there is a link '¿Ya tienes cuenta?' (Do you already have an account?) and a green button labeled 'Registrarse' (Register). The footer of the form states '© 2026 Canela's Desk. Todos los derechos reservados.' (© 2026 Canela's Desk. All rights reserved.).

Figura 26. Pantalla de Registro.

## Perfil

Dentro del Perfil, el usuario puede acceder a diferentes apartados:

- **Información del perfil:** permite cambiar el avatar, visualizar los datos personales y consultar información relacionada con la isla.
- **Actualización de contraseña:** opción para modificar la contraseña actual.
- **Eliminación de cuenta:** botón ubicado en la parte inferior que permite eliminar la cuenta de usuario.

### Información del perfil

Personaliza tu identidad y los datos de tu isla.

**Tu Avatar**



**CAMBIAR AVATAR**  
Haz clic para elegir un vecino

---

**Nombre**

**Email**

**Biografía**

**Hemisferio**

Sur ▾

**Nombre de la Isla**

**Fruta de la Isla**

Melocotones 🍑 ▾

Guardar Cambios

### Actualizar contraseña

Usa una contraseña larga y segura para proteger tu cuenta.

**Contraseña actual**

**Nueva contraseña**

**Confirmar contraseña**

Guardar

Figura 27. Pantalla de Perfil. Aquí el usuario puede ver y cambiar sus datos.

### Eliminar cuenta

Una vez eliminada tu cuenta, todos tus datos se borrarán permanentemente. Descarga cualquier información que desees conservar antes de proceder.

ELIMINAR CUENTA

Figura 28. Botón para eliminar la cuenta del usuario, este se encuentra en el Perfil.

## 5 Pruebas

### 5.1 Prueba de Usabilidad:

Para esta prueba de usabilidad probaremos como el usuario interactúa con la aplicación web. No se evalúan los conocimientos del usuario, sino la facilidad de uso del sistema.

- **Tarea:** Regístrate y añade un vecino a tus favoritos.
- **Duración aproximada:** 2 minutos

Pasos que siguió el usuario:

**Paso 1:** El usuario accede al sitio web y localiza la pestaña “Vecinos” en el menú de navegación.

**Paso 2:** Al entrar en la sección Vecinos, puede ver los detalles de cada personaje, pero no hay opción para guardarlos como favoritos.



Figura 29. Pantalla de Inicio que ve el usuario.

**Paso 3:** El usuario selecciona el botón “Identifícate” y elige la opción de Registro para crear una cuenta.

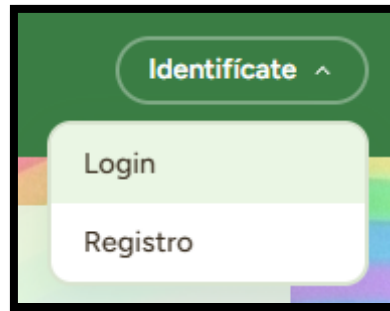


Figura 30. Botón con un desplegable para Login y Registro.

**Paso 4:** Completa el formulario con nombre, correo y contraseña, y supera el captcha para finalizar el registro.

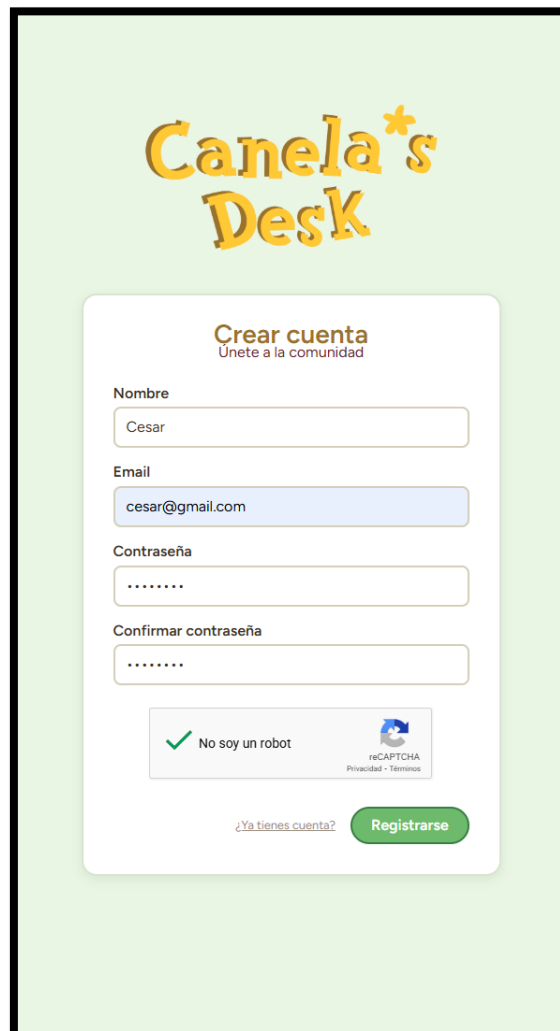
A registration form titled 'Crear cuenta' with the subtitle 'Únete a la comunidad'. The form has four input fields: 'Nombre' (containing 'Cesar'), 'Email' (containing 'cesar@gmail.com'), 'Contraseña' (masked with dots), and 'Confirmar contraseña' (masked with dots). Below the fields is a CAPTCHA section with a green checkmark and the text 'No soy un robot'. At the bottom, there is a link '¿Ya tienes cuenta?' and a green button labeled 'Registrarse'.

Figura 31. Usuario registrándose.

**Paso 5:** Una vez iniciado sesión, se muestran nuevas pestañas disponibles en la interfaz.



Figura 32. Barra de navegación que ve el usuario al registrarse.

**Paso 6:** El usuario ingresa a la pestaña “Vecinos”, donde ahora cada tarjeta cuenta con un ícono de corazón que permite agregar personajes a favoritos.

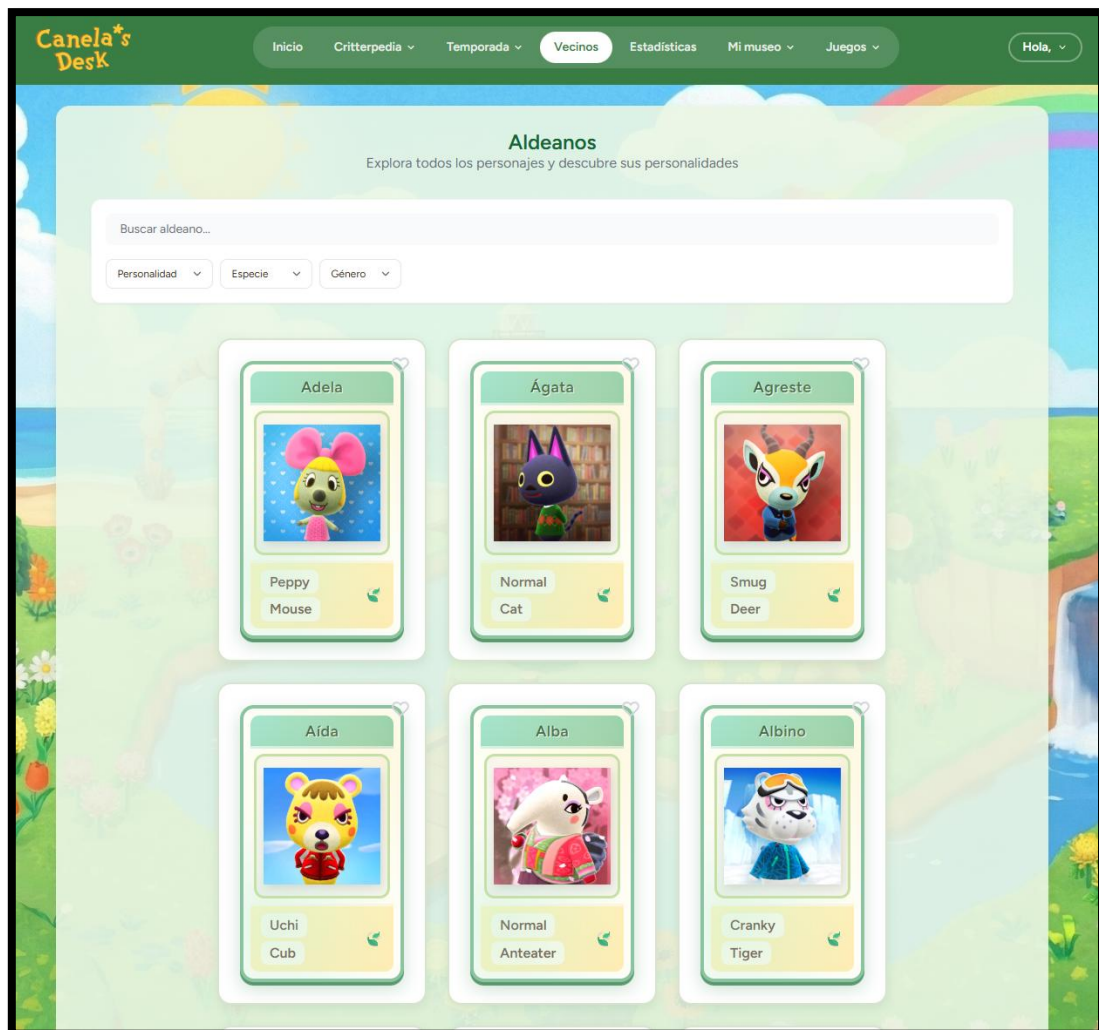


Figura 33. Pestaña de Vecinos en la que el usuario debe guardar su colección.

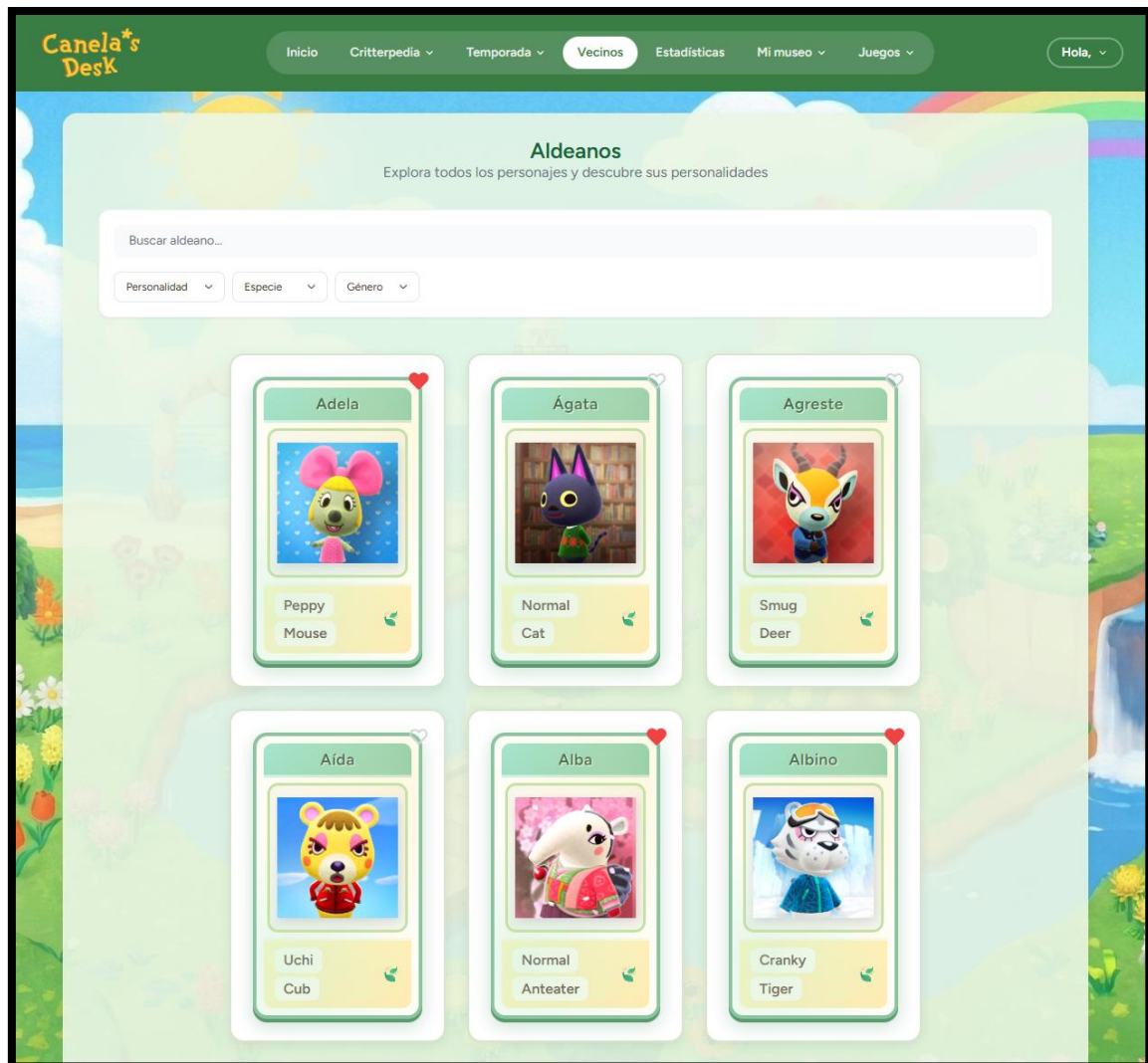


Figura 34 Usuario añadiendo personajes a su colección.

**Paso 7:** Al ingresar a la sección “Mi museo”, específicamente en la pestaña Vecinos, se confirma que los personajes agregados como favoritos se han guardado correctamente.

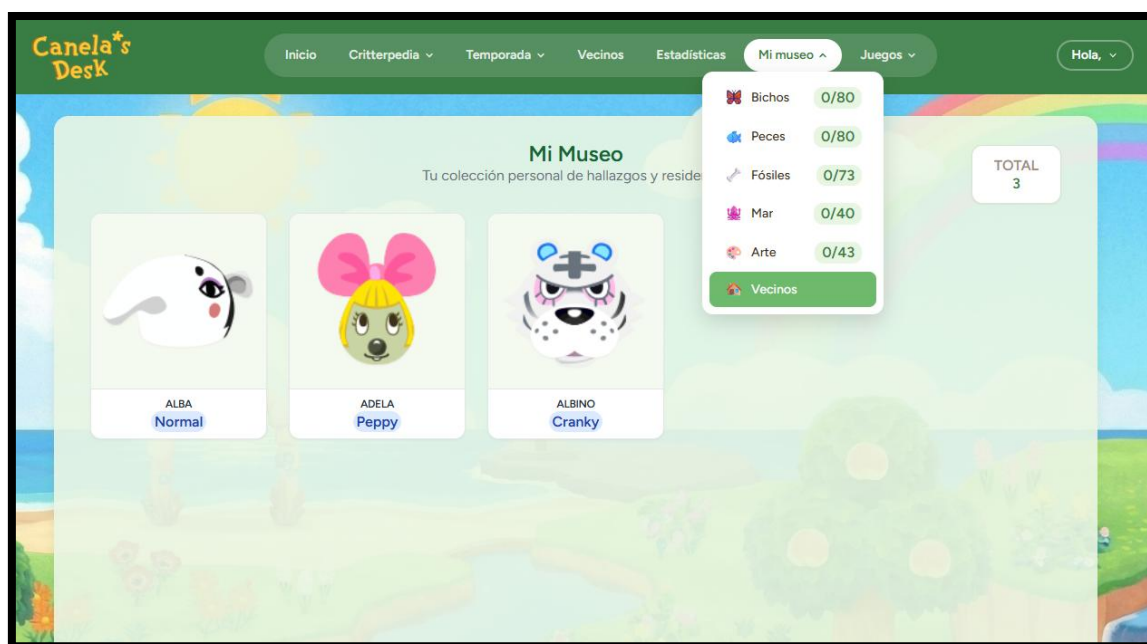


Figura 35. Pestaña de Museo con los vecinos que guardo en su colección.

## 5.2 Prueba de Accesibilidad con WAVE:

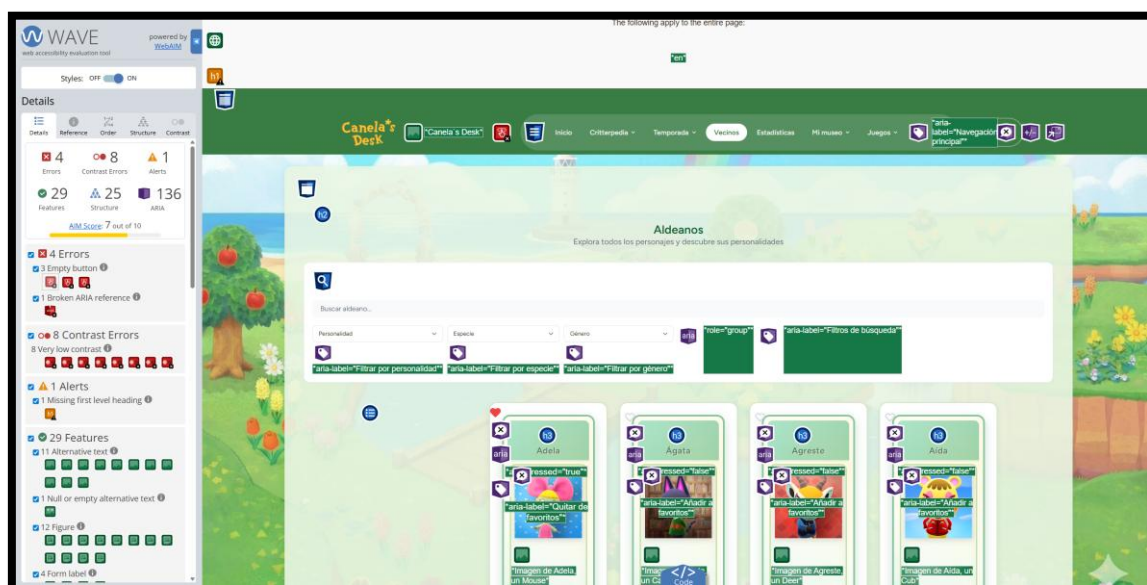


Figura 36. Prueba de Accesibilidad con WAVE.

En esta sección se evalúa la calidad de la interacción del usuario con la plataforma "Canela's Desk", analizando la estructura de navegación y el cumplimiento de estándares de accesibilidad web.

### 5.2.1 Análisis de Navegación y UI



El diseño de la interfaz se ha centrado en la claridad y la facilidad de uso, basándose en los siguientes puntos clave:

- **Navegación Intuitiva:** El menú principal permite un acceso rápido a las secciones de la Critterpedia, Temporada y Estadísticas, manteniendo una jerarquía clara.
- **Gestión de Categorías:** En la sección "Mi Museo", se utiliza un sistema de pestañas con iconos visuales y contadores dinámicos que informan al usuario sobre su progreso de colección de un solo vistazo.
- **Feedback Inmediato:** El uso de etiquetas aria-label en los filtros de búsqueda (personalidad, especie, género) asegura que el usuario comprenda la función de cada control antes de interactuar.

### 5.2.2 2. Evaluación de Accesibilidad (Test WAVE)

Para garantizar una web inclusiva, se ha sometido la aplicación a una auditoría con la herramienta WAVE (Web Accessibility Evaluation Tool), obteniendo los siguientes resultados:

- **Cumplimiento de Estándares ARIA:** Se han implementado con éxito 136 etiquetas ARIA para facilitar la navegación con lectores de pantalla, incluyendo roles de grupo y descripciones detalladas para los botones de favoritos.
- **Textos Alternativos:** El sistema incluye atributos alt descriptivos en las imágenes de los aldeanos y coleccionables (ej. "Imagen de Adela, un Mouse"), asegurando que la información visual sea accesible para usuarios con discapacidad visual.
- **Puntos de Mejora Identificados:** La auditoría señala áreas de optimización técnica, como la resolución de 4 errores de estructura y la mejora del contraste en ciertos elementos de texto para cumplir estrictamente con los niveles de conformidad WCAG.

### 5.2.3 Conclusión de Usabilidad

La arquitectura de la información, apoyada por un diagrama de casos de uso que define claramente las interacciones permitidas para visitantes y usuarios registrados, garantiza una curva de aprendizaje mínima. La integración de elementos semánticos de HTML5 y etiquetas de accesibilidad posiciona a "Canela's Desk" como una herramienta robusta y funcional para cualquier perfil de jugador.



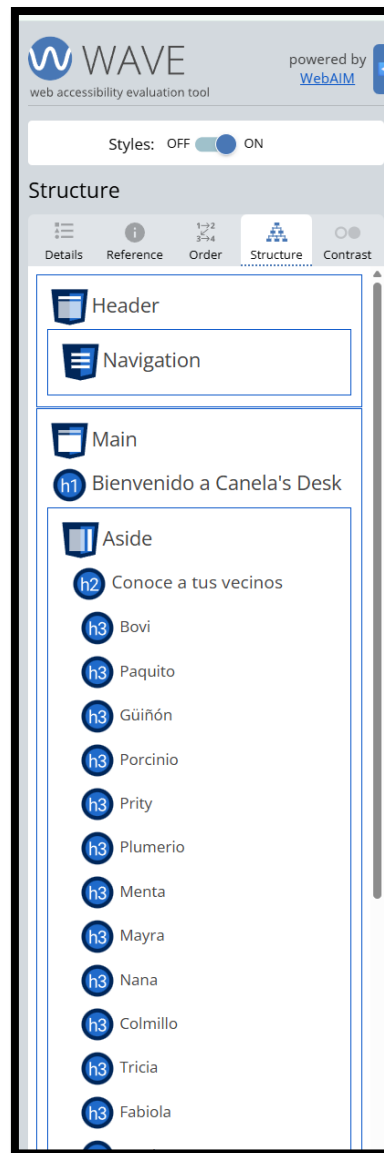


Figura 37. Pestaña de WAVE con la estructura del sitio web.

## 5.3 Testing

Para garantizar la robustez, seguridad y estabilidad de Canela's Desk, se ha implementado una metodología de pruebas automatizadas cubriendo los aspectos críticos del sistema. El objetivo principal ha sido validar que cada funcionalidad responda correctamente ante diversos escenarios, asegurando un producto final libre de errores lógicos.

### 5.3.1 Resumen de ejecución de pruebas

Se ha utilizado el framework PHPUnit 11.5 para la ejecución de una suite de pruebas integrales. Los resultados obtenidos confirman la integridad del software bajo un entorno de pruebas controlado.

Tabla 3. Resumen de ejecución de pruebas.

| Dato                      | Valor                        |
|---------------------------|------------------------------|
| Framework de testing      | PHPUnit 11.5.49              |
| Versión de PHP            | 8.3.28 (entorno Laragon)     |
| Base de datos de pruebas  | SQLite(ejecución en memoria) |
| Total de tests Ejecutados | 61                           |
| Total de Assertions       | 138                          |
| Resultado final           | PASSED (0 errores, 0 fallos) |
| Tiempo de ejecución       | ~3.9 segundos                |

### 5.3.2 Metodología y entorno

Las pruebas se han dividido en dos niveles para asegurar una cobertura completa:

- **Feature Tests (Pruebas de Funcionalidad):** Simulan el comportamiento del usuario final interactuando con las rutas y endpoints de la aplicación.
- **Unit Tests (Pruebas Unitarias):** Verifican la lógica interna de los servicios y modelos de forma aislada.

### 5.3.3 Desglose de Pruebas por Módulo

#### A. Autenticación y Seguridad (18 tests)

Se ha verificado el ciclo de vida completo del usuario:

- Registro de nuevos usuarios y validación de formularios.
- Inicio y cierre de sesión seguro.
- Verificación de correo electrónico mediante URLs firmadas.
- Procesos de recuperación y actualización de contraseñas.
- Protección de rutas privadas (Middleware Auth).

#### B. Catálogo y API (7 tests)

Validación de los puntos de acceso de datos para todas las colecciones (Peces, Bichos, Arte, Fósiles y Criaturas Marinas):

- Respuestas correctas en formato JSON.
- Funcionamiento de filtros por rareza y búsqueda por nombre.
- Lógica de ordenación por precio (ascendente/descendente).

#### C. Sistema de Donaciones y Museo (7 tests)

Pruebas sobre la lógica de colección personal:

- Funcionalidad de "Toggle": Marcar y desmarcar objetos como donados.
- Sincronización de estadísticas del museo en tiempo real.

- **Gestión de la Caché:** Verificación de que los datos se almacenan y se invalidan correctamente para optimizar el rendimiento.

#### D. Experiencia de Usuario y Juegos (9 tests)

- **Juego de Memoria:** Validación de la generación de niveles, control de tiempos y sistema de récords (solo se actualiza el perfil si la nueva puntuación supera a la anterior).
- **Interfaz:** Renderizado correcto de la página de bienvenida tanto para invitados como para usuarios registrados.

#### 5.3.4 Depuración y Validación del Esquema de Datos

El proceso de pruebas no solo validó la estabilidad del software, sino que fue fundamental para documentar y asegurar la integridad de los datos heredados de la ACNH API (fuente externa sin mantenimiento, integrada localmente).

- **Integridad Estructural:** La construcción de los datos de prueba (factories) permitió confirmar la obligatoriedad de campos críticos del esquema original, como *file\_name* (identificador único) y *birthday\_string* (formato de fecha de aldeanos), garantizando que la base de datos mantenga la fidelidad con la fuente original.
- **Normalización de Atributos:** Se identificaron y documentaron discrepancias en la nomenclatura de precios. Mientras que la mayoría de los catálogos emplean un campo genérico *price*, la categoría de Arte requiere *buy\_price* y *sell\_price* según la estructura de la API. Estas particularidades quedaron correctamente mapeadas en los recursos de la aplicación para asegurar una respuesta coherente en el frontend.
- **Seguridad y Aislamiento:** Debido a la integración de Google *ReCaptcha*, se implementó un *mock* global del servicio de validación. Esto permitió ejecutar las pruebas de autenticación de forma aislada, sin dependencia de servicios externos, verificando además la protección de rutas mediante el control de códigos de estado 401 *Unauthorized*.

#### 5.3.5 Conclusión del Proceso

La suite de pruebas funcionó como una red de seguridad técnica. Su implementación permitió blindar el sistema ante la complejidad del esquema de datos heredado, garantizando que cualquier modificación futura respete la estructura completa y la lógica de negocio establecida.

## 6 Despliegue

### 6.1 Instrucciones de instalación en local:

Este proyecto utiliza Docker para garantizar un entorno de ejecución idéntico independientemente del sistema operativo del host. A continuación, se detallan los pasos para la puesta en marcha del sistema.

### 6.1.1 Requisitos previos

Para instalar y ejecutar la aplicación, es necesario disponer de:

- **Docker Desktop** (con soporte para WSL2 en Windows).
- **Node.js y npm** (para la compilación inicial del frontend).
- **Git** (para la gestión del repositorio).

### 6.1.2 Pasos de instalación

- Clonación y preparación del entorno

En primer lugar, se debe clonar el repositorio y configurar las variables de entorno necesarias para la conexión de los contenedores:

```
git clone [URL_DEL_REPOSITORIO]
cd Backend
cp .env.example .env
```

Nota: Es fundamental verificar que DB\_HOST=db y REDIS\_HOST=redis en el archivo .env.

- Construcción y levantamiento de contenedores

Se utiliza Docker Compose para orquestrar los cuatro servicios (PHP, Nginx, MySQL y Redis):

```
docker-compose up -d --build
```

Este comando descargará las imágenes necesarias y configurará el servidor web en el puerto 8000.

- Instalación de dependencias y configuración de Laravel

Una vez los contenedores están en estado *Up*, se ejecutan los comandos internos de configuración de la aplicación:

# Instalación de librerías de PHP

```
docker exec -it canela-app composer install
```

# Generación de la clave de seguridad de la aplicación

```
docker exec -it canela-app php artisan key:generate
```

# Creación del enlace simbólico para archivos multimedia

```
docker exec -it canela-app php artisan storage:Link
```

- Migración y carga de datos (Seeding)

Para poblar la base de datos con la información de los JSON (bichos, peces y aldeanos), se ejecuta:

```
docker exec -it canela-app php artisan migrate --seed
```

- Compilación del Frontend

Para que la interfaz reactiva sea visible, se deben compilar los activos de Vue.js:

```
npm install
npm run build
```

### 6.1.3 Verificación del despliegue

Tras completar los pasos anteriores, la aplicación estará disponible en la dirección: <http://localhost:8000>

El sistema de logs se puede monitorizar mediante:

```
docker logs -f canela-app (Para errores de PHP/Laravel).
docker logs -f canela-nginx (Para peticiones HTTP).
```

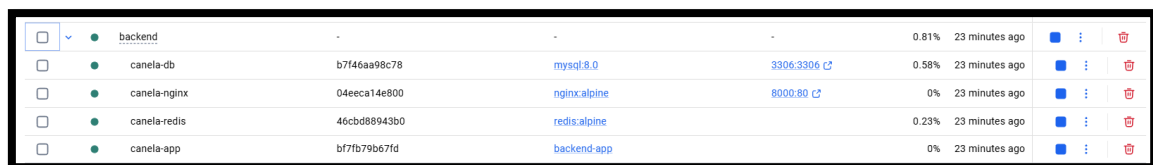
### 6.1.4 Descripción de los archivos de configuración

Para que la arquitectura de contenedores funcione, se han redactado cuatro archivos fundamentales que orquestan el sistema:

- **Dockerfile:** Es la receta para construir la imagen de la aplicación. Define el sistema operativo base (PHP 8.2-fpm), instala las extensiones necesarias para que Laravel conecte con MySQL y Redis, e integra Composer para la gestión de dependencias.
- **docker-compose.yml:** Es el director de orquesta. Define los cuatro servicios (App, Servidor Web, Base de Datos y Caché) y establece cómo se comunican entre ellos mediante una red interna privada. También gestiona los "volúmenes", que permiten que los cambios que haces en tu código se reflejen en tiempo real dentro del contenedor.

- **docker-compose/nginx/default.conf:** Es el archivo de reglas del servidor web. Le dice a Nginx cómo interpretar las peticiones del navegador, dónde buscar los archivos de Vue (en la carpeta public) y cómo enviar el código PHP al contenedor de la aplicación para que lo procese.
- **.env:** Es el archivo de identidad del proyecto. Contiene las "llaves" y nombres de la base de datos que permiten que Laravel sepa que ya no está en un servidor local tradicional (como Laragon), sino que debe buscar el servicio de base de datos en el host db dentro de la red de Docker.

### 6.1.5 Resultado final:



| Container    | ID           | Image        | Port Mapping | CPU Usage | Uptime         |
|--------------|--------------|--------------|--------------|-----------|----------------|
| backend      | -            | -            | -            | 0.81%     | 23 minutes ago |
| canela-db    | b7f46aa98c78 | mysql:8.0    | 3306:3306    | 0.58%     | 23 minutes ago |
| canela-nginx | 04eeca14e800 | nginx:alpine | 8000:80      | 0%        | 23 minutes ago |
| canela-redis | 46cbd88943b0 | redis:alpine | -            | 0.23%     | 23 minutes ago |
| canela-app   | bf7fb79b67fd | backend-app  | -            | 0%        | 23 minutes ago |

Figura 38. Contenedores de Docker.

### 6.1.6 URLs:

- Del **repositorio** de Git.

<https://github.com/Cesarauneg/Proyecto-Intermodular>

- Del sitio **web** puesto en producción (si se ha desplegado).

## 7 Sostenibilidad y "Green Coding"

### 7.1 Aplicación Sin desplegar

El desarrollo de Canela's Desk se ha regido por principios de Green IT, buscando la máxima eficiencia de recursos. La optimización no solo mejora la experiencia de usuario (UX), sino que reduce directamente el consumo eléctrico del servidor y las emisiones de carbono asociadas.

#### 7.1.1 Estrategia de caché

##### *Estrategia de Caché Multinivel*

Dado que la información del juego (criaturas, fósiles y música) procede de un dataset inmutable, se ha implementado una estrategia de caché agresiva. Esto evita la computación redundante y el estrés sobre el motor de base de datos MySQL.

Tabla 4 Estrategia de persistencia y TTL por tipo de dato

| Dato cacheado           | TTL      | Consultas evitadas (por acceso)           |
|-------------------------|----------|-------------------------------------------|
| Catálogos completos     | 1 hora   | 1 SELECT masivo (80+ registros por tipo)  |
| Música horaria          | 24 horas | 1 SELECT (48 pistas de audio)             |
| Filtros de aldeanos     | 24 horas | 4 SELECT DISTINCT complejos               |
| Estadísticas de usuario | 5 min    | 5 consultas COUNT con JOIN (Tablas pivot) |

Además, mediante el uso de Seeders, se han eliminado las dependencias de APIs externas en tiempo de ejecución. Esto suprime la latencia de red y el consumo energético derivado de peticiones HTTP constantes a servidores de terceros.

#### Optimización de Recursos Frontend

El ecosistema de herramientas de Vite 7 permite que la aplicación sea extremadamente ligera:

- Minificación: Compresión de activos mediante esbuild y Lightning CSS.
- Code Splitting: Gracias a Inertia.js, el cliente solo descarga el JavaScript estrictamente necesario para la vista actual.
- Lazy Loading nativo: Las imágenes solo se transfieren cuando entran en el viewport del usuario, ahorrando ancho de banda.
- Arquitectura SPA: Al intercambiar componentes Vue sin recargar el HTML base, se reduce drásticamente el volumen de datos transferidos por sesión.

#### Eficiencia de Infraestructura: Docker y Redis

En el entorno de producción, la arquitectura se eleva mediante contenedores independientes. La principal mejora es la sustitución de la caché de ficheros por Redis (In-memory storage):

Tabla 5 Comparativa de rendimiento: Sistema de archivos local frente a Redis en Docker

| Métrica              | Caché en Fichero (Local) | Caché en Redis (Docker) |
|----------------------|--------------------------|-------------------------|
| Tiempo de respuesta  | 2 – 5 ms                 | < 1 ms                  |
| Acceso a disco (I/O) | Sí (Desgaste físico)     | No (Todo en RAM)        |
| Escalabilidad        | Limitada al nodo         | Horizontal y compartida |

El uso de Docker permite limitar los recursos de CPU y RAM por contenedor, garantizando que el servidor trabaje en su punto de máxima eficiencia energética sin picos innecesarios.

### *Cuantificación del Ahorro y Equivalencias*

Considerando un escenario de 1.000 visitas diarias a la página de inicio, la reducción de carga es del 95%. Los cálculos se basan en un factor de emisión medio de 0,25 kg CO<sub>2</sub>/kWh.

*Tabla 6 Análisis comparativo de consumo energético y emisiones de CO<sub>2</sub>.*

| Concepto          | Sin Optimizar              | Con Optimización          | Ahorro Estimado              |
|-------------------|----------------------------|---------------------------|------------------------------|
| Consultas SQL/día | 10.000                     | 500                       | 9.500 consultas/día          |
| Energía servidor  | 1,1 kWh/año                | 0,06 kWh/año              | ~1 kWh/año                   |
| Emisiones Totales | ~220,27 kg CO <sub>2</sub> | ~0,014 kg CO <sub>2</sub> | ~220 kg CO <sub>2</sub> /año |

### *Impacto en el mundo real*

Para poner en perspectiva el ahorro de 220 kg de CO<sub>2</sub> al año, estas son sus equivalencias:

- Movilidad: Conducir un coche de combustión durante 900 km.
- Tecnología: Cargar un smartphone de forma diaria durante 60 años.
- Naturaleza: El trabajo de 11 árboles adultos absorbiendo CO<sub>2</sub> durante un año entero.
- Hogar: Mantener una bombilla LED encendida de forma ininterrumpida durante 10 años

## 8 Conclusiones

### 8.1 Autoevaluación

#### 8.1.1 Dificultades encontradas:

El desarrollo de Canela's Desk no ha estado exento de retos técnicos, los cuales han servido como oportunidades de aprendizaje crítico:

- **Gestión de Entornos con Docker:** Uno de los mayores desafíos fue la transición de un entorno local tradicional (Laragon) a una arquitectura de contenedores. Surgieron conflictos con la persistencia de datos en MySQL y errores de permisos en el servidor Nginx (como el error



ERR\_EMPTY\_RESPONSE), que requirieron una investigación profunda sobre la configuración de volúmenes y redes internas de Docker.

- **Consumo y Mapeo de la API:** La API original de *Animal Crossing* proporcionaba datos en formatos a veces inconsistentes. Se tuvo que diseñar un sistema de *Seeders* robusto en Laravel para procesar los archivos JSON, filtrar la información irrelevante y normalizarla en nuestra base de datos para asegurar un rendimiento óptimo.
- **Sincronización de Versiones (Git):** Durante el trabajo en equipo, surgieron conflictos de fusión (*merge conflicts*) en archivos críticos como el *composer.lock* o el archivo de rutas. Esto obligó a establecer una política de ramas más estricta para evitar la pérdida de código.
- **Conexión Backend-Frontend con Inertia.js:** Al ser una tecnología que elimina la necesidad de una API REST tradicional, el aprendizaje de la comunicación entre los controladores de Laravel y los componentes de Vue 3 supuso una curva de aprendizaje inicial pronunciada.

### 8.1.2 Valoración de la metodología.

Para este proyecto se ha seguido una metodología Ágil, adaptada a las necesidades de un entorno educativo y de desarrollo rápido.

- **Coordinación y Comunicación:** El equipo ha mantenido una comunicación constante, utilizando herramientas de control de versiones (GitHub) como eje central. La división de tareas se realizó de forma modular: mientras una parte del equipo se centraba en la lógica del servidor y los modelos de datos, la otra trabajaba en la reactividad de la interfaz y el diseño visual.
- **Gestión de Tareas:** Se han realizado reuniones de seguimiento para evaluar el progreso y reasignar recursos en los puntos donde el desarrollo se estancaba (como en el despliegue de Docker). Esta flexibilidad permitió que el proyecto no se detuviera ante bloqueos técnicos.
- **Control de Calidad:** La metodología de "Revisiones de Código" informales ayudó a detectar errores de lógica de forma temprana y a asegurar que el estilo visual fuera consistente en todas las vistas de la aplicación.

## 8.2 Líneas futuras

Aunque "Canela's Desk" es una herramienta funcional y completa, existen diversas líneas de desarrollo que podrían convertir la aplicación en un producto comercializable y altamente competitivo dentro de la comunidad de jugadores.

### 8.2.1 Gamificación y Economía Interna (Sistema de Bayas)

Para aumentar la retención de usuarios, se propone la implementación de un ecosistema económico basado en el juego original:

- **Implementación de una Tienda Virtual:** Creación de un catálogo de artículos (como los discos de Totakeke) que los usuarios podrían "comprar" para personalizar su perfil.
- **Sistema de Minijuegos:** Desarrollo de pequeños juegos interactivos en la plataforma (ej: adivinar el pez por su sombra) que premien al usuario con Bayas (moneda virtual).
- **Retos Diarios:** Recompensas por registrar capturas o visitar la aplicación varios días seguidos, fomentando el uso recurrente.

### 8.2.2 Optimización de la Experiencia de Usuario (UX/UI)

- **Modo Oscuro Dinámico:** Implementación de una interfaz de alto contraste diseñada para el uso nocturno. Esta estética no solo mejora la accesibilidad, sino que se alinearía con el Green Coding, reduciendo el consumo de batería en dispositivos con pantallas OLED.
- **Notificaciones Push:** Sistema de alertas que avise al usuario cuando un bicho o pez "raro" esté a punto de desaparecer al finalizar el mes, evitando que pierda la oportunidad de completarlo en su museo.
- **Sincronización con Nintendo Switch:** Investigación de APIs de terceros o reconocimiento de imágenes mediante IA para que el usuario pueda subir una captura de pantalla de su consola y la app detecte automáticamente qué vecinos tiene o qué fósiles ha donado.

### 8.2.3 Funciones Sociales y Comunidad

- **Intercambio de "Deseados":** Una sección donde los usuarios puedan publicar qué aldeanos están buscando y contactar con otros jugadores que los tengan en su isla y planeen mudarlos.
- **Ranking de Museos:** Tablas de clasificación globales para ver quién ha completado más secciones del museo, fomentando una competición sana entre la comunidad.
- **Exportación de "Pasaporte":** Generación de una imagen personalizada del perfil del usuario (estilo pasaporte de ACNH) para ser compartida en redes sociales (Instagram/Twitter), funcionando como marketing orgánico para la aplicación.

### 8.2.4 Hacia un producto comercializable

Para que la app sea un producto de mercado, se requeriría:

- **Versión Mobile Nativa:** Transformar la web actual en una PWA (Progressive Web App) o aplicación nativa para que funcione sin conexión a internet en los momentos de juego.

- **Soporte Multi-idioma:** Adaptar todos los datos (nombres de peces, bichos y frases de Sócrates) a los diferentes idiomas disponibles en el videojuego para alcanzar una audiencia global.

#### 8.2.5 Reflexión Final

En conclusión, Canela's Desk ha evolucionado de ser un simple catálogo de datos a convertirse en una plataforma integral que demuestra la potencia del ecosistema Laravel + Vue.js.

A pesar de los desafíos técnicos iniciales especialmente en la orquestación de entornos con Docker y la normalización de datos externos, el equipo ha logrado construir una arquitectura sólida, escalable y con una experiencia de usuario fluida gracias a Inertia.js. Este proyecto no solo cumple con los objetivos técnicos planteados, sino que sienta las bases para una herramienta comunitaria capaz de aportar valor real a los jugadores de Animal Crossing. Más allá del código, el desarrollo de esta aplicación ha sido un testimonio del aprendizaje adaptativo y de la eficacia de las metodologías ágiles en la resolución de problemas complejos.

## 9 Bibliografía

- [1] **WebAIM**, «WAVE web accessibility evaluation tool,» 2026. [En línea]. Available: <https://wave.webaim.org/>. [Último acceso: febrero 2026].
- [2] **Laravel Team**, «Laravel: The PHP Framework for Web Artisans (v12.x),» 2026. [En línea]. Available: <https://laravel.com/docs/12.x>. (Documentación oficial para el desarrollo del backend, Eloquent ORM y Starter Kits).
- [3] **Vue.js Core Team**, «Vue.js - The Progressive JavaScript Framework (v3.x),» 2026. [En línea]. Available: <https://vuejs.org/>. (Guía oficial para la implementación de Composition API).
- [4] **Inertia.js**, «The Modern Monolith - Inertia.js Documentation,» 2026. [En línea]. Available: <https://inertiajs.com/>.
- [5] **Docker Inc.**, «Docker Documentation and Best Practices,» 2026. [En línea]. Available: <https://docs.docker.com/>. (Guía para la orquestación de servicios con Docker Compose).
- [6] **Tailwind Labs**, «Tailwind CSS v3.4 Documentation,» 2026. [En línea]. Available: <https://tailwindcss.com/docs>. (Manual de utilidades CSS para el diseño responsive).
- [7] **Redis**, «Redis Documentation and Data Structures,» 2026. [En línea]. Available: <https://redis.io/docs/>. (Capa de caché de alto rendimiento).
- [8] **Discord Inc.**, «Discord - Product and Communication Safety,» 2026. [En línea]. Available: [enlace sospechoso eliminado]. (Comunicación y coordinación del equipo).
- [9] **GitHub**, «GitHub Docs - Version Control and Collaboration,» 2026. [En línea]. Available: <https://docs.github.com/>. (Gestión de repositorio).
- [10] **A. Lours**, «ACNHAPI - Animal Crossing New Horizons API Source,» 2024. [En línea]. Available: <https://github.com/alexislours/ACNHAPI>. (Fuente de datos JSON para criaturas y vecinos).
- [11] **Google**, «Gemini 1.5 Flash - Large Language Model for AI Collaboration,» 2026. [En línea]. Available: <https://gemini.google.com>. (Consultoría de infraestructura y redacción técnica).
- [12] **Anthropic**, «Claude 3.5 Sonnet & Claude Code - Advanced Coding Assistant,» 2026. [En línea]. Available: <https://www.anthropic.com/claude>. (Generación de lógica compleja y refactorización).
- [13] **OpenAI**, «ChatGPT (o1/GPT-4o) - Conversational AI for Development,» 2026. [En línea]. Available: <https://chat.openai.com>. (Resolución de sintaxis SQL y soporte en diseño de BD).
- [14] **Nintendo Co., Ltd.**, «Animal Crossing: New Horizons - Official Site,» 2026. [En línea]. Available: <https://www.animal-crossing.com/new-horizons/>. (Referencia visual y de marca).

- [15] **Chart.js**, «Simple yet flexible JavaScript charting (v4.5),» 2026. [En línea]. Available: <https://www.chartjs.org>.
- [16] **Laragon**, «Laragon: Entorno de desarrollo universal para Windows,» 2026. [En línea]. Available: <https://laragon.org>. (Prototipado inicial).
- [17] **Laravel Team**, «Laravel Breeze & Sanctum,» 2026. [En línea]. Available: <https://laravel.com/docs/12.x>. (Sistemas de autenticación y seguridad de API).
- [18] **Mockery Team**, «Mockery: PHP Mock Object Framework (v1.7),» 2026. [En línea]. Available: <https://docs.mockery.io>.
- [19] **PHPUnit Team (S. Bergmann)**, «PHPUnit: The PHP Testing Framework (v11.5),» 2026. [En línea]. Available: <https://docs.phpunit.de>.
- [20] **Tighten Co.**, «Ziggy: Use your Laravel routes in JavaScript (v2.x),» 2026. [En línea]. Available: <https://github.com/tighten/ziggy>.
- [21] **Catdad (Kiril Vatev)**, «Canvas-confetti (v1.9),» 2026. [En línea]. Available: <https://github.com/catdad/canvas-confetti>.
- [22] **Vite Team**, «Vite: Next Generation Frontend Tooling (v7.x),» 2026. [En línea]. Available: <https://vite.dev>.
- [23] **Vue-chartjs**, «Vue-chartjs: Vue.js wrapper for Chart.js (v5.3),» 2026. [En línea]. Available: <https://vue-chartjs.org>.
- [24] **Google**, «reCAPTCHA: Seguridad y protección contra fraude,» 2026. [En línea]. Available: <https://developers.google.com/recaptcha>.